Malware Analysis - Lab Notes

IMPORTANT WARNING

- These notes have been made starting from my notes taken during lessons and the power points published by the professor, then rearranged through the help of Claude to produce a narrative flow.
- All the content belongs to the Professor of the course.
- While I try to ensure everything is inside these notes, please don't use them as a "source of truth", and study from official sources as well.

CHAPTER 1: x86 Assembly

x86 Assembly for Malware Analysis — Reading Compiled C

Reverse engineering a malicious binary almost always begins with the same problem: the source code is gone, and all that remains is machine code. Before any payload, persistence mechanism, or network protocol can be understood, an analyst must be able to look at a stream of x86 instructions and reconstruct the high-level logic the compiler started from. The skill is pattern recognition. Compilers are deterministic translators, so every `if`, every `while`, and every function call leaves a recognizable fingerprint in the assembly. Learning those fingerprints on clean, well-understood code is what makes it possible to read hostile code later, where there are no comments, no symbols, and often deliberate attempts to mislead.

This chapter builds that foundation using a small 32-bit Windows program compiled from a set of ordinary C control-flow functions. The binary targets the 80386 instruction set — the first 32-bit Intel CPU and still the baseline for a huge amount of real-world malware — and is examined statically in a disassembler.

1. The Analysis Environment

Malware analysis is performed inside a disposable virtual machine, restored from a clean snapshot before each session. The guiding principle is isolation: the machine should contain nothing of value and should have no casual path to the outside world. A sample that is being dissected must never be given the opportunity to

reach the host, the local network, or the internet, because even a sample intended only for study can execute, spread, or exfiltrate if handled carelessly. Keeping the VM behind a restrictive network configuration and treating every binary as live ordnance is the baseline discipline of the field.

The primary tool is a disassembler. A **disassembler** translates raw machine-code bytes back into human-readable assembly mnemonics; a full-featured one such as IDA goes further, performing automatic analysis to recover function boundaries, cross-references, and data types, and it also embeds a **debugger** so the same binary can later be executed one instruction at a time. Static disassembly (reading the code without running it) and dynamic debugging (watching it run) are the two complementary halves of analysis. A free edition of IDA is sufficient for everything in this chapter.

1.1 Loading a Portable Executable

When a Windows binary is opened, the disassembler first identifies the file format. A 32-bit Windows program is reported as a *Portable Executable for 80386 (PE)*. Because nothing in the raw code declares how functions receive their arguments, the tool must assume a default **calling convention**, and for this platform that default is `__cdecl` — the convention used by virtually all C code compiled for 32-bit Windows. This assumption drives how the tool interprets stack accesses as function arguments, so it matters from the very first instruction.

1.2 Symbols: a luxury that real malware denies

A compiled binary can carry **debug information** — a side table mapping machine addresses back to original function names, variable names, and line numbers. The GCC/MinGW toolchain emits this information in the **DWARF** format (the standard used by Linux compilers, and by GCC even when it targets Windows). When such information is present, the disassembler offers to load it, and the result is dramatic: functions appear with their original names (`fun_if`, `fun_while`, `main`) and global variables appear as `A`, `B`, `C` instead of bare addresses.

This convenience is almost never available in practice. An author of malicious code has every incentive to **strip** the binary, removing all symbols so that functions appear only as generic labels like `sub_401460` and variables only as hexadecimal addresses. The same program can be built in both a symbol-rich form and a stripped form; comparing them is the fastest way to appreciate how much interpretive work symbols do, and how much of that work falls back onto the

analyst when they are absent. For this reason, the patterns taught below must ultimately be recognizable *without* any names at all — the names are training wheels.

There is one recurring exception worth remembering: a careless author sometimes leaves behind the path to the program database (PDB) file that was generated at build time. That leftover string can reveal the directory structure, user name, and project layout of the machine on which the malware was compiled — a small intelligence gift from a sloppy adversary.

1.3 Annotations are not code

The disassembler decorates its output with inferred commentary: reconstructed function prototypes, argument names, stack-frame summaries, and structural hints such as `; __unwind {`. These lines are **analysis output, not instructions that exist in the binary**. One of the first discriminations an analyst must learn is separating what the CPU will actually execute from what the tool merely believes about it. A related discrimination is separating the handful of functions the author wrote from the large body of **compiler and runtime support code** that every C program is linked against (startup routines, C library internals, thread-local-storage callbacks). Time is wasted analyzing library code; identifying and ignoring it is a skill in itself.

Finally, the disassembler offers two views of the same code. The **text view** is a linear listing; the **graph view** arranges the code into **basic blocks** — straight-line runs of instructions with a single entry and a single exit — connected by arrows that represent possible control flow. A conditional branch produces two outgoing edges, conventionally drawn so that the *condition-true* and *condition-false* successors are visually distinct. The graph view is the natural way to see the shape of an `if` or a loop at a glance, and it is where the constructs below become obvious.

2. Anatomy of a Function

Everything that follows depends on understanding how a function uses the **stack**, so this section establishes the model once.

2.1 Global variables

The program declares three global integers, `A`, `B`, and `C`. Globals live at fixed addresses in a data section for the entire lifetime of the process, so every reference

to them is an absolute memory access. In symbol-rich disassembly these appear as `ds:A`, `ds:B`, `ds:C`; stripped, they would appear as `ds:dword_407074` and similar. The key recognition cue is the *absolute address* operand — a hallmark of a global, as opposed to the frame-relative operands used for locals.

2.2 The stack frame, prologue, and epilogue

Local variables and function arguments are not stored at fixed addresses; they live on the stack, in a region called the function's **stack frame**. To address that region reliably, a function anchors it with the **base pointer** register `ebp`. Nearly every function therefore begins with the same two-instruction **prologue**:

```
push    ebp             ; save the caller's base pointer
mov     ebp, esp        ; set our base pointer to the current
stack top
```

The `push` writes the caller's `ebp` to the stack (growing the stack downward by four bytes) so it can be restored later; the `mov` then freezes `ebp` at the current top of stack. From this instant, `ebp` is a stable reference point for the whole function, even as `esp` moves around. A function that needs space for locals then subtracts from `esp` (for example `sub esp, 0x10` reserves sixteen bytes).

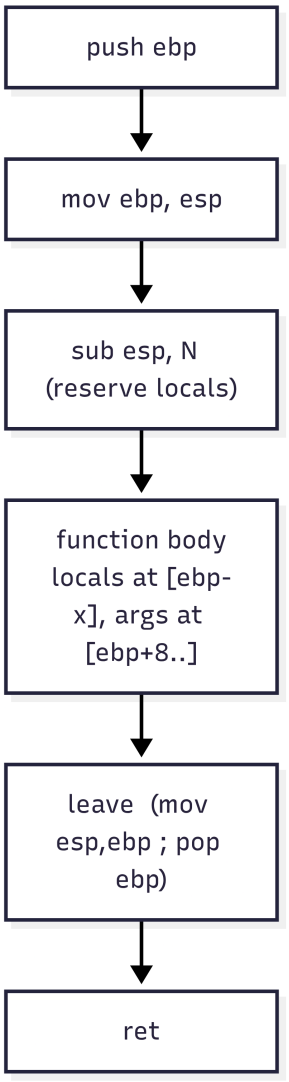
With `ebp` fixed, the two sides of the frame have opposite signs:

- **Local variables** sit *below* the base pointer, at **negative** offsets: `[ebp-4]`, `[ebp-8]`, `[ebp-0xC]`, and so on. A variable named by convention `var_4` is simply "the variable at offset 4 below `ebp`." Because these accesses are everywhere, the pattern `[ebp - something]` should immediately read as "local variable" even before its meaning is known. The disassembler lets the analyst assign a name and a size (a 32-bit integer is a `dword`) to each such slot and have every reference updated at once — a core manual-annotation step.
- **Arguments and the return address** sit *above* the base pointer, at **positive** offsets. Immediately after the prologue, `[ebp]` holds the saved old `ebp`, `[ebp+4]` holds the return address pushed by the `call` that entered the function, and `[ebp+8]`, `[ebp+0xC]`, `[ebp+0x10]` hold the first, second, and third arguments respectively.

The mirror image of the prologue is the **epilogue**, which tears the frame down and returns:

```
leave                   ; equivalent to: mov esp, ebp ; pop ebp
ret                     ; pop return address and jump to it
```

The single-byte `leave` instruction restores `esp` and `ebp` in one step; `ret` then transfers control back to the caller. A function that allocated no locals may instead end with a bare `pop ebp` / `ret`. Recognizing prologue and epilogue is what lets an analyst carve an unknown binary into functions in the first place.



2.3 The `__cdecl` calling convention

Under `__cdecl`, arguments are passed on the stack and pushed by the **caller** in right-to-left order, so that the leftmost argument ends up at the lowest address (`[ebp+8]`) inside the callee. Crucially, under `__cdecl` the **caller is responsible for cleaning the arguments off the stack** after the call returns. The function's integer result is returned in the `eax` register. These three facts — arguments on the stack, caller-side cleanup, result in `eax` — are the signature of `__cdecl`, and

because it is the default for this platform, they describe almost every hand-written function here.

3. The Driver: `main`

The program's `main` is a thin driver that sets the globals and calls each demonstration function in turn. Its disassembly establishes several patterns at once:

```
push    ebp
mov     ebp, esp
and     esp, 0FFFFFFF0h     ; align stack to a 16-byte boundary
call    ___main             ; MinGW C-runtime initialization
mov     ds:A, 1Eh           ; A = 30
mov     ds:B, 15h           ; B = 21
mov     ds:C, 3             ; C = 3
call    _fun_if
call    _fun_if_else
call    _fun_while
call    _fun_do_while
call    _fun_for
mov     eax, 0              ; return 0
leave
retn
```

Two details deserve comment. The instruction `and esp, 0FFFFFFF0h` clears the low four bits of the stack pointer, forcing it to a multiple of sixteen; this is **stack alignment**, required by the platform's ABI for performance and for certain instructions, and it is a normal part of a `main` prologue rather than anything suspicious. The call to `___main` is **MinGW runtime initialization** — compiler support code, not program logic — and is exactly the kind of boilerplate the analyst learns to recognize and step past. The constants `1Eh`, `15h`, and `3` are the decimal values 30, 21, and 3 written in hexadecimal, the native radix of disassembly; fluent hex reading is non-negotiable.

4. Recognizing Control-Flow Constructs

The heart of the chapter is translating conditional and iterative C back out of assembly. Assembly has no `if` or `while`; it has only **comparisons** that set the

`EFLAGS` register and **conditional jumps** that act on those flags. The compiler builds every high-level construct from this pair, and it does so following consistent templates.

The single most important idea is **condition inversion**. High-level code reads *"if the condition holds, execute the body."* The compiler finds it more convenient to generate *"if the condition does **not** hold, jump **past** the body."* As a result, the comparison in the assembly almost always tests the **logical opposite** of the condition written in C: a source-level `if (A < x)` becomes an assembly `jge` (jump if greater-or-equal) that *skips* the body. Reading the jump and mentally negating it to recover the original condition is a reflex the analyst must build.

4.1 The `if` statement — `fun_if`

The source computes a simple guarded subtraction:

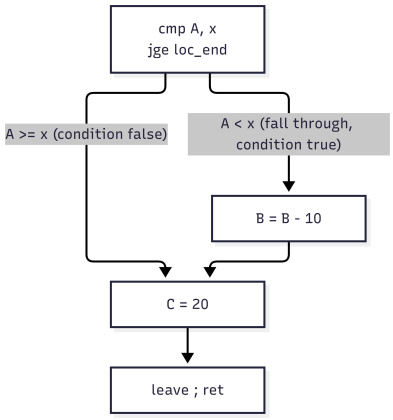
```
void fun_if() {
    int x = 30;
    if (A < x) {
        B = B - 10;
    }
    C = 20;
}
```

Its disassembly:

```
push    ebp
mov     ebp, esp
sub     esp, 10h
mov     [ebp-4], 1Eh        ; x = 30            (local var_4)
mov     eax, ds:A
cmp     eax, [ebp-4]        ; compare A with x
jge     short loc_end       ; if A >= x, skip the body   <--
inverted condition
mov     eax, ds:B           ; --- body: B = B - 10 ---
sub     eax, 0Ah
mov     ds:B, eax
loc_end:
mov     ds:C, 14h           ; C = 20
```

```
nop
leave
ret
```

The local `x` is materialized at `[ebp-4]` and set to `30`. The comparison `cmp eax, [ebp-4]` evaluates `A - x` only to set flags. The source condition is `A < x`; the compiler emits its negation, `jge` (jump if `A >= x`), to **jump over** the body when the body should *not* run. If the jump is not taken, control falls through into the body, which loads `B`, subtracts ten (`0Ah`), and stores it back. Either way, execution converges on the final `C = 20` (`14h`) and the epilogue. This convergence — two paths meeting at a common successor — is the structural signature of an `if`.



4.2 The `if / else` statement — `fun_if_else`

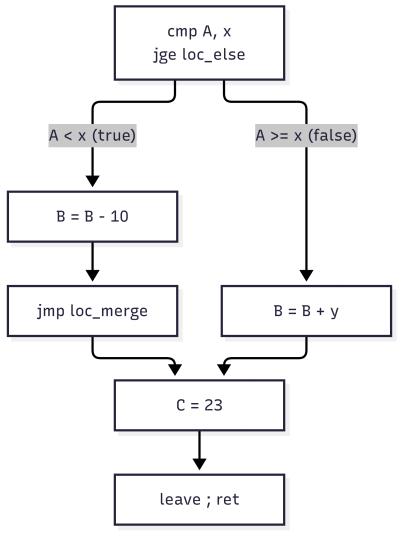
Adding an `else` introduces a second body and a mandatory unconditional jump:

```
void fun_if_else() {
    int x = 30, y = 5;
    if (A < x) {
        B = B - 10;
    } else {
        B = B + y;
    }
    C = 23;
}
```

```
sub     esp, 10h
mov     [ebp-4], 1Eh        ; x = 30
mov     [ebp-8], 5          ; y = 5
```

```
mov     eax, ds:A
cmp     eax, [ebp-4]
jge     short loc_else      ; if A >= x, go to the else branch
mov     eax, ds:B           ; --- then: B = B - 10 ---
sub     eax, 0Ah
mov     ds:B, eax
jmp     short loc_merge     ; skip over the else branch
loc_else:
mov     edx, ds:B           ; --- else: B = B + y ---
mov     eax, [ebp-8]
add     eax, edx
mov     ds:B, eax
loc_merge:
mov     ds:C, 17h           ; C = 23
```

The inverted `jge` now selects between two real alternatives rather than skipping one. The decisive new element is the `jmp short loc_merge` at the end of the *then* branch: once the true path finishes, it must leap over the *else* path, because otherwise execution would fall straight into code meant only for the false case. That **unconditional jump at the tail of the first branch, landing on the same block the second branch falls into**, is the distinguishing mark of an `if/else` as opposed to a bare `if`. Both branches reunite at `loc_merge`, where the shared tail (`C = 23`, i.e. `17h`) runs.



4.3 The `while` loop — `fun_while`

Loops reveal a second compiler habit: the **condition test is placed at the bottom** of the loop, and the function jumps straight to that test on entry.

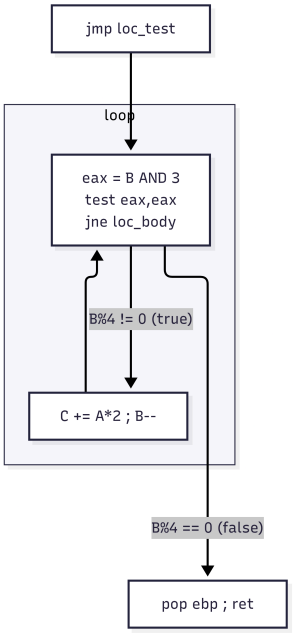
```
void fun_while() {
    while (B % 4 != 0) {
        C += A * 2;
        B--;
    }
}
```

```
push    ebp
mov     ebp, esp
jmp     short loc_test      ; jump straight to the condition
loc_body:
mov     eax, ds:A
lea     edx, [eax+eax]      ; edx = A * 2
mov     eax, ds:C
add     eax, edx
mov     ds:C, eax           ; C = C + A*2
mov     eax, ds:B
sub     eax, 1
mov     ds:B, eax           ; B = B - 1
loc_test:
mov     eax, ds:B
and     eax, 3              ; eax = B % 4   (keep the low two
bits)
test    eax, eax            ; is the remainder zero?
jne     short loc_body      ; if non-zero, loop again
nop
pop     ebp
ret
```

Three idioms appear together here. First, the structure is **test-at-the-bottom with an initial jump to the test**: the very first thing the function does is `jmp loc_test`, so the condition is evaluated before the first iteration, exactly matching `while` semantics (the body may run zero times). The back-edge is the conditional jump `jne loc_body` that climbs back up to the body when the loop should continue. A backward-pointing conditional jump is the universal signature of a loop.

Second, `B % 4` is computed not with a division instruction but with `and eax, 3`. Because four is a power of two, the remainder modulo four is simply the value of the **low two bits**, and `and` with the mask `3` (binary `11`) isolates exactly those bits. The subsequent `test eax, eax` checks whether that result is zero. This "mask with (2^n − 1) to take a modulo by 2^n" trick is extremely common and should be read directly as a modulo operation.

Third, `A * 2` is computed with `lea edx, [eax+eax]` rather than a multiply — the subject of §5.



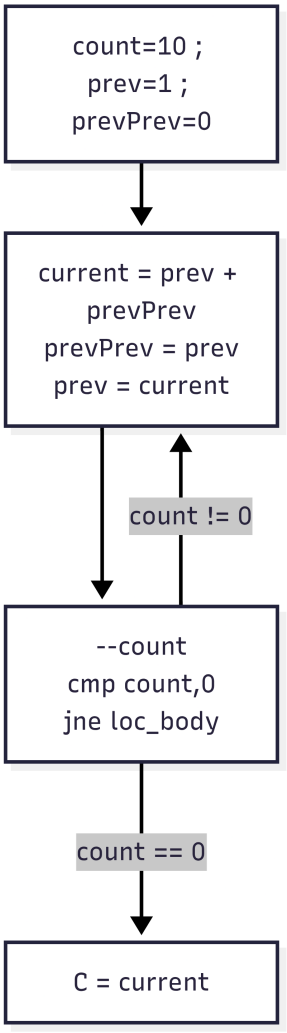
4.4 The `do / while` loop — `fun_do_while`

A `do/while` differs from a `while` in exactly one structural way: there is **no initial jump to the test**, because the body must always run at least once.

```
void fun_do_while() {
    int count = 10;
    int current, prev = 1, prevPrev = 0;
    do {
        current = prev + prevPrev;
        prevPrev = prev;
        prev = current;
    } while (--count);
    C = current;
}
```

```
sub     esp, 10h
mov     [ebp-4], 0Ah        ; count = 10
mov     [ebp-8], 1          ; prev = 1
mov     [ebp-0Ch], 0        ; prevPrev = 0
loc_body:
mov     edx, [ebp-8]        ; prev
mov     eax, [ebp-0Ch]      ; prevPrev
add     eax, edx
mov     [ebp-10h], eax      ; current = prev + prevPrev
mov     eax, [ebp-8]
mov     [ebp-0Ch], eax      ; prevPrev = prev
mov     eax, [ebp-10h]
mov     [ebp-8], eax        ; prev = current
sub     [ebp-4], 1          ; --count
cmp     [ebp-4], 0
jne     short loc_body      ; while (count != 0)
mov     eax, [ebp-10h]
mov     ds:C, eax           ; C = current
```

Execution enters the body directly; only after the body does it decrement `count` and test it with a backward `jne`. The absence of a leading `jmp` to the condition is precisely what separates `do/while` from `while` at the assembly level. This routine also illustrates how multiple locals are packed into the frame — `count`, `prev`, `prevPrev`, and `current` occupy `[ebp-4]`, `[ebp-8]`, `[ebp-0Ch]`, and `[ebp-10h]` — and how a classic Fibonacci-style sliding-window computation looks once reduced to register shuffling.



4.5 The `for` loop — `fun_for`

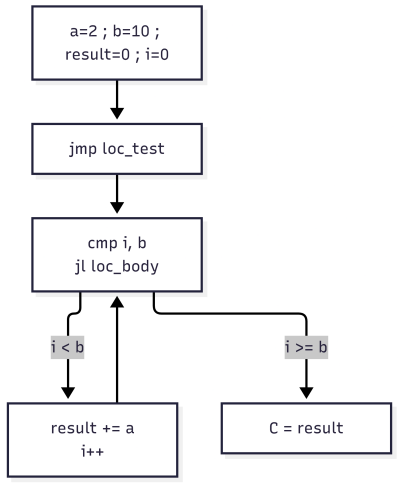
A `for` loop is, at the machine level, a `while` loop with its four parts (initialization, condition, body, increment) laid out in a fixed arrangement:

```
void fun_for() {
    int a = 2, b = 10;
    int i, result = 0;
    for (i = 0; i < b; i++) {
        result += a;
    }
    C = result;
}
```

```
sub     esp, 10h
mov     [ebp-0Ch], 2        ; a = 2
mov     [ebp-10h], 0Ah      ; b = 10
mov     [ebp-8], 0          ; result = 0
```

```
mov     [ebp-4], 0          ; i = 0           (initialization)
jmp     short loc_test      ; jump to the condition
loc_body:
mov     eax, [ebp-0Ch]
add     [ebp-8], eax        ; result += a     (body)
add     [ebp-4], 1          ; i++             (increment)
loc_test:
mov     eax, [ebp-4]
cmp     eax, [ebp-10h]
jl      short loc_body      ; i < b ?         (condition)
mov     eax, [ebp-8]
mov     ds:C, eax           ; C = result
```

The layout matches the `while` template exactly — an initial `jmp` to a bottom-placed test, a backward conditional jump as the back-edge — with one addition: the **increment is emitted at the end of the body**, right before control reaches the test. Recognizing that the last statement of the loop body is a counter update (`add [ebp-4], 1`) separated from the user's actual work (`result += a`) is what lets an analyst reconstruct a `for` rather than an indistinct `while` . Note also that the condition here is `i < b` , and the emitted jump is `jl` (jump if less) — a case where the compiler kept the condition *un-inverted* because the jump target is the body rather than the exit. Whether the condition is inverted depends on whether the jump guards the body or skips it, and the analyst determines that from the jump's target, not from a fixed rule.



5. The `LEA` Instruction

Several of the routines above compute values with `lea` , an instruction whose behavior is a frequent source of confusion and therefore a favorite of both

compilers and obfuscators.

`LEA` **(Load Effective Address)** evaluates a memory-address expression and writes the resulting *address* into a destination register. The essential distinction is this: ordinary data-movement and arithmetic instructions that use a memory operand **dereference** it — they read or write the contents of the pointed-to location. `LEA` does **not** touch memory at all. It performs only the address *calculation* implied by the brackets and stores that number. In other words, where `mov eax, [ebx+ecx*4]` fetches the value *at* `ebx + ecx*4`, `lea eax, [ebx+ecx*4]` merely computes `ebx + ecx*4` and keeps it.

This gives `LEA` two distinct uses:

The first is its literal purpose, **address calculation for pointer arithmetic** — computing the address of an array element or structure field without reading it, for example to pass a pointer to another function.

The second, and the reason it appears in arithmetic-heavy code, is as a **fast general-purpose arithmetic instruction**. Because the x86 addressing mode can encode `base + index*scale + displacement` in a single operation — with scale factors of 1, 2, 4, or 8 — `LEA` can evaluate small linear expressions in one instruction, without disturbing the flags register. Thus `lea edx, [eax+eax]` computes `eax * 2` (seen in `fun_while` for `A * 2`), `lea eax, [ecx+ebx]` computes the sum `ecx + ebx`, and `lea eax, [eax+eax*4]` computes `eax * 5`. When an analyst sees `LEA` with an arithmetic-looking bracket expression and the result is subsequently used as a *number* rather than as a pointer, it should be read as plain arithmetic. Confusing the two readings — treating a computed constant as a pointer, or vice versa — is one of the most common early mistakes in reading disassembly.

5bis. Analyst's Reference: Pattern-to-Construct Cues

The following cues summarize the fingerprints established in this chapter. They are stated in terms that survive stripping, since in real analysis the names are gone.

A **prologue** `push ebp` / `mov ebp, esp` (optionally followed by `sub esp, N`) marks a function entry; a matching `leave` / `ret` or `pop ebp` / `ret` marks its exit. Operands of the form `[ebp - k]` are **local variables**; operands `[ebp +`

`8]`, `[ebp + 0Ch]`, `[ebp + 10h]` are **incoming arguments**; operands that are **absolute addresses** are **globals**.

A `cmp` or `test` followed by a **conditional jump** is a decision point. If the jump **skips a block** and both paths reconverge, it is an **`if`**; the tested condition is the *inverse* of the source condition. If the first block ends in an **unconditional `jmp` over a second block** and both reconverge, it is an **`if/else`**. A **backward** conditional jump is a **loop back-edge**: if the function **jumps to the condition first** and the body is below, it is a **`while`** (or a `for`, if the body's last action is a counter increment); if the body runs **before** any test, it is a **`do/while`**.

Arithmetic shortcuts recur constantly: `and reg, (2^n - 1)` is a **modulo by 2^n**; a `shl` / `shr` is a multiply/divide by a power of two; and `lea` with a bracketed expression used as a number is **linear arithmetic**, not a memory access. Hexadecimal constants should be converted on sight (`0Ah` = 10, `14h` = 20, `1Eh` = 30).

Finally, and most importantly: the symbol names, inferred prototypes, and structural comments supplied by the disassembler are **hypotheses to verify, not facts**. In a genuine investigation they will be absent or deliberately misleading, and the only reliable ground truth is the instruction stream itself.

6. Arrays and Indexed Memory Access

An array is a contiguous block of equally sized elements, so accessing element i reduces to a single address computation: `element_address = base + i * element_size`. This maps directly onto the x86 `base + index*scale` addressing mode, where the **scale** is the size of one element. Because the scale is encoded in the instruction, the disassembly reveals the element size for free: a scale of `4` means four-byte elements (`int`, `float`, or a 32-bit pointer), `1` means bytes (`char`), `2` means 16-bit shorts, and `8` means eight-byte elements. Reading the scale is the quickest way to infer the type of the data being iterated.

The routine `fun_array` adds a local array to a global array element by element:

```
int A[5] = { 1, 2, 3, 4, 5 };   // global array
int X, Y;
void fun_array() {
    int v[5] = { 6, 7, 8, 9, 10 };   // local array
    int i;
    for (i = 0; i < sizeof(A)/sizeof(int); ++i)
```

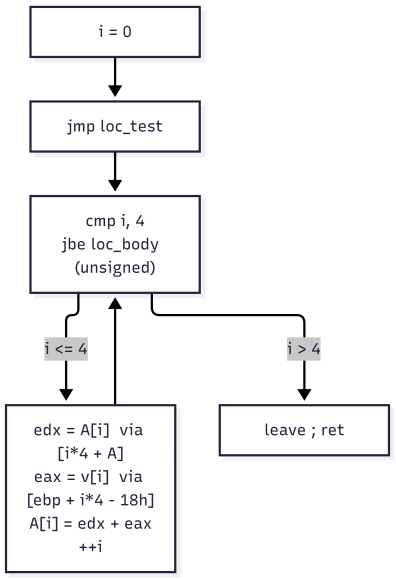
```
        A[i] += v[i];
}
```

```
push    ebp
mov     ebp, esp
sub     esp, 20h
mov     [ebp-18h], 6        ; v[0] = 6     (local array begins at
ebp-18h)
mov     [ebp-14h], 7        ; v[1] = 7
mov     [ebp-10h], 8        ; v[2] = 8
mov     [ebp-0Ch], 9        ; v[3] = 9
mov     [ebp-8],   0Ah      ; v[4] = 10
mov     [ebp-4],   0        ; i = 0
jmp     short loc_test
loc_body:
mov     eax, [ebp-4]
mov     edx, [eax*4 + A]        ; edx = A[i]   (global base,
scale 4)
mov     eax, [ebp-4]
mov     eax, [ebp + eax*4 - 18h] ; eax = v[i]  (frame base ebp-
18h, scale 4)
add     edx, eax
mov     eax, [ebp-4]
mov     [eax*4 + A], edx        ; A[i] = A[i] + v[i]
add     [ebp-4], 1             ; ++i
loc_test:
mov     eax, [ebp-4]
cmp     eax, 4
jbe     short loc_body        ; continue while i <= 4   (UNSIGNED
compare)
nop
leave
ret
```

Two addressing patterns stand side by side and should be read as "array element access" on sight. The **global array** `A` is reached as `[eax*4 + A]` — an *absolute base address* plus the index scaled by four. The **local array** `v` is reached as `[ebp + eax*4 - 18h]` — a *frame-relative base* (`ebp - 18h` , the address of `v[0]`)

plus the same scaled index. The presence of an index register multiplied by a scale is the giveaway that a single value is being selected out of a sequence; whether the base is an absolute address or an `ebp`-relative offset tells you whether the array is global or local.

The loop bound carries a subtle but important lesson. In source, the condition is `i < sizeof(A)/sizeof(int)`. The expression `sizeof(A)/sizeof(int)` equals `20/4 = 5` and is a **compile-time constant**, so no size calculation appears at run time — the compiler has already folded it. What *does* appear is `cmp eax, 4` followed by **`jbe`** (jump if below-or-equal), and `jbe` is an **unsigned** conditional jump. The compiler emitted an unsigned comparison because `sizeof` yields a value of the unsigned type `size_t`, which forces the whole comparison to be unsigned. Whenever the disassembly shows the unsigned branch family — `ja`, `jae`, `jb`, `jbe` — rather than the signed family — `jg`, `jge`, `jl`, `jle` — it is a strong hint that the operands are unsigned quantities such as sizes, lengths, counts, or pointers. The bound also shifted form: the source `i < 5` became the machine test `i <= 4`, an equivalent the compiler is free to choose.



7. The `switch` Statement

A `switch` is a multi-way branch on a single integer. Compilers implement it with one of two strategies, and the choice depends on how the case labels are distributed. Recognizing which strategy was used — and reversing it back into the original cases — is a frequent exam task and a frequent real-world one, because dispatch logic (command handlers, C2 opcode parsers, state machines) in malware is very often a large `switch`.

7.1 Strategy one: the comparison ladder (sparse cases)

When the case values are few or spread far apart, the compiler emits a straightforward chain of compare-and-jump instructions — effectively an `if / else if / else` ladder. The routine `fun_switch_simple` switches on the global `X` over just two explicit cases:

```
void fun_switch_simple() {
    int result;
    switch (X) {
        case 1:  result = 7; break;
        case 2:  result = 6; break;
        default: result = 5;
    }
    Y = result;
}
```

```
mov     eax, ds:X
cmp     eax, 1
je      short case_1
cmp     eax, 2
je      short case_2
jmp     short case_default
case_1:
mov     [ebp-4], 7
jmp     short loc_merge
case_2:
mov     [ebp-4], 6
jmp     short loc_merge
case_default:
mov     [ebp-4], 5
loc_merge:
mov     eax, [ebp-4]
mov     ds:Y, eax           ; Y = result
```

Structurally this is identical to an `if/else` chain, and that is exactly the point: at the assembly level a sparse `switch` and a hand-written `if/else if` ladder are indistinguishable. Two features identify it as multi-way selection on one variable. First, the *same* value (here `X`, loaded once into `eax`) is compared against a series of constants. Second, each case body ends with a `jmp` to a

common **merge** point — the compiled form of the `break` statement. The shared destination is where control reconverges before `Y = result` runs. If a `break` were omitted in source, the corresponding `jmp` would simply be absent and execution would **fall through** into the next case's body — a pattern worth watching for, since fall-through is both a common bug and a deliberate idiom.

7.2 Strategy two: the jump table (dense cases)

When the case values are many and form a compact, contiguous range, comparing them one by one is wasteful. Instead the compiler builds a **jump table**: an array of code addresses, one per case, stored in read-only data. The switch value is used as an index into this table, and a single indirect jump dispatches directly to the correct case in constant time, regardless of how many cases there are. The routine `fun_switch_complex` switches over the contiguous range `0..4`:

```
void fun_switch_complex() {
    int result;
    switch (X) {
        case 0: result = 7; break;
        case 1: result = 6; break;
        case 2: result = 5; break;
        case 3: result = 4; break;
        case 4: result = 3; break;
        default: result = 8;
    }
    Y = result;
}
```

```
mov     eax, ds:X
cmp     eax, 4
ja      short case_default        ; if (unsigned)X > 4  ->
default
jmp     ds:jpt[eax*4]             ; indirect jump through the
jump table

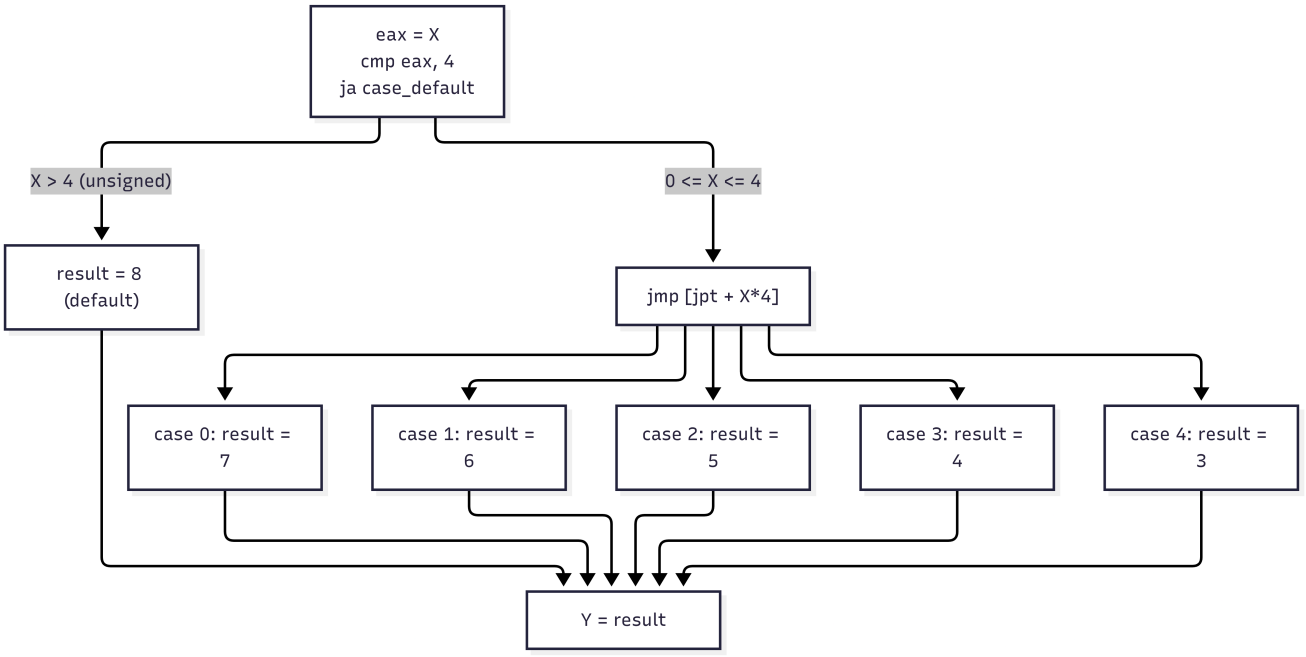
; jpt (in read-only data) holds five code pointers:
;   jpt[0] -> case_0   (result = 7)
;   jpt[1] -> case_1   (result = 6)
```

```
;   jpt[2] -> case_2   (result = 5)
;   jpt[3] -> case_3   (result = 4)
;   jpt[4] -> case_4   (result = 3)

case_0: mov [ebp-4], 7 ; jmp short loc_merge
case_1: mov [ebp-4], 6 ; jmp short loc_merge
case_2: mov [ebp-4], 5 ; jmp short loc_merge
case_3: mov [ebp-4], 4 ; jmp short loc_merge
case_4: mov [ebp-4], 3 ; jmp short loc_merge
case_default:
mov     [ebp-4], 8
loc_merge:
mov     eax, [ebp-4]
mov     ds:Y, eax           ; Y = result
```

Three elements define this pattern and should be memorized as the signature of a jump table. First, a **bounds check**: `cmp eax, 4` followed by `ja case_default`. The use of `ja` (jump if *above*, the unsigned comparison) is deliberate and elegant — a single unsigned test rejects both values greater than the maximum index *and* negative values, because a negative number reinterpreted as unsigned becomes enormous and therefore also "above" the bound. One comparison thus funnels every out-of-range input to the default case. Second, the **indirect jump** `jmp ds:jpt[eax*4]`: the validated index is scaled by four (the size of a 32-bit code pointer) and used to fetch an address from the table, to which control then transfers. An indirect `jmp` whose target is read from `base + index*4` is almost always a compiled `switch`. Third, the **table itself**, a contiguous array of code addresses living in read-only data, each entry pointing at the first instruction of one case body. As before, every case ends with a `jmp` to the shared merge block, the compiled `break`.

A capable disassembler recognizes this idiom, reconstructs the table, and labels each target as a case; without that assistance — in stripped or deliberately obfuscated code — the analyst reads the table manually, following each stored pointer to recover the cases by hand. The deciding factor between the two strategies is case density: a tight contiguous range favors the constant-time table, while sparse or widely separated labels favor the comparison ladder (and very large sparse sets may be compiled into a balanced binary search over the values).



8. Calling Conventions in Depth

A calling convention is the contract between a caller and a callee: where arguments are placed, in what order, who removes them from the stack afterward, which registers must be preserved, and where the return value is delivered. Because the machine code on each side is generated independently — often by different compilers, or linked in from separate libraries — both sides must agree on the convention or the stack will be corrupted. For an analyst, reading the convention off the disassembly is immediately useful: it reveals how many arguments a function takes, helps distinguish the author's own functions from imported API calls, and, when a convention is hand-rolled or violated, can itself betray hand-written assembly or shellcode.

Three conventions account for nearly all 32-bit Windows code, and all three return a 32-bit integer result in `eax`. They differ mainly in **who cleans the arguments off the stack**.

8.1 `cdecl` — the C default

Under `cdecl`, arguments are passed on the stack right-to-left and the **caller** removes them after the call returns. This caller-side cleanup is what makes variadic functions such as `printf` possible, since only the caller knows how many arguments were actually supplied. The callee of `mysum(int a, int b)`:

```
_mysum:
push    ebp
```

```
mov     ebp, esp
mov     eax, [ebp+8]        ; first argument  a
mov     edx, [ebp+0Ch]      ; second argument b
add     eax, edx            ; return value left in eax
nop
pop     ebp
ret                         ; plain ret -> the caller will clean
the arguments
```

The arguments are read from `[ebp+8]` and `[ebp+0Ch]`, the sum is left in `eax`, and the function ends with a **plain `ret`** that does not touch the arguments. The single most reliable convention cue lives exactly here: **a bare `ret` means the caller cleans the stack (`cdecl`); a `ret N` carrying an immediate means the callee cleaned `N` bytes itself.**

The caller, `test`, shows the complementary half, with a modern-compiler twist:

```
push    esi
push    ebx
sub     esp, 18h
mov     esi, 20h            ; a = 32   (held in a register:
'register int')
mov     ebx, 3Ch            ; b = 60
mov     [esp+4], ebx        ; place 2nd arg into reserved
outgoing-arg space
mov     [esp], esi          ; place 1st arg
call    _mysum
mov     [ebp-0Ch], eax      ; save the result
lea     eax, [esi+ebx]      ; expected = a + b
mov     ds:expected, eax
...
add     esp, 18h            ; tear down the frame (also reclaims
the arg space)
```

The canonical textbook form of a `cdecl` call is "push the arguments, `call`, then `add esp, N` to discard them." What appears here is the optimized realization that current compilers prefer: rather than a `push` / `add esp` pair around every call, the function reserves a block of **outgoing-argument space** once (`sub esp,`

`18h`) and writes each argument into it with `mov [esp+k]`. The cleanup is then **folded into the epilogue** (`add esp, 18h`), which reclaims arguments and locals together. Both are `cdecl`; the analyst must recognize that the absence of an `add esp, 8` immediately after a call does not contradict caller-side cleanup — it has merely been deferred and merged into the frame teardown. This routine also illustrates the `register` storage hint: `a` and `b` live in `esi` and `ebx` instead of on the stack, and `a + b` is computed with `lea eax, [esi+ebx]`.

8.2 `stdcall` — the Windows API convention

Under `stdcall`, arguments are still pushed right-to-left, but the **callee** removes them before returning, using `ret N`, where `N` is the total size of the arguments in bytes. This is the convention of essentially the entire Win32 API, so recognizing it is central to reading any program that calls Windows functions. The three-argument `mysum`:

```
_mysum:
push    ebp
mov     ebp, esp
mov     edx, [ebp+8]        ; a
mov     eax, [ebp+0Ch]      ; b
add     edx, eax
mov     eax, [ebp+10h]      ; c
add     eax, edx            ; a + b + c in eax
pop     ebp
ret     0Ch                 ; callee pops 12 bytes (3 args) ->
stdcall
```

The decisive instruction is `ret 0Ch`: the function removes twelve bytes — its three four-byte arguments — from the stack as it returns. A `ret` with a non-zero immediate is the fingerprint of a callee-cleanup convention, and the immediate itself discloses the argument footprint: `ret 0Ch` means twelve bytes, that is, three dword parameters. The caller correspondingly performs **no** post-call `add esp`:

```
sub     esp, 28h
mov     [esp+8], 60h        ; 96   (3rd arg)
mov     [esp+4], 40h        ; 64   (2nd arg)
mov     [esp],   20h        ; 32   (1st arg)
call    _mysum
```

```
sub     esp, 0Ch            ; re-reserve the 12 bytes the callee
popped
mov     [ebp-0Ch], eax      ; result
```

Because this compiler again pre-reserves its outgoing-argument block, the callee's `ret 0Ch` leaves `esp` twelve bytes higher than the caller's frame bookkeeping expects, so the compiler re-subtracts (`sub esp, 0Ch`) to restore the reserved region. The direction of this adjustment is the mirror image of `cdecl`: where a `cdecl` caller *adds* to `esp` to discard arguments, a `stdcall` caller here *subtracts* to compensate for arguments the callee has already discarded. Either way the stable rule holds — the `ret N` in the callee is the authoritative sign of `stdcall`.

8.3 Returning data through pointers

Not every result comes back in `eax`. A function that must return more than one value, or that writes into storage owned by the caller, instead receives the **address** of that storage as an argument and writes through it — the compiled form of a C output parameter (`int* result`). The routine `mysum(int a, int b, int* result)`:

```
_mysum:
push    ebp
mov     ebp, esp
mov     edx, [ebp+8]        ; a
mov     eax, [ebp+0Ch]      ; b
add     edx, eax            ; a + b
mov     eax, [ebp+10h]      ; result   (a pointer)
mov     [eax], edx          ; *result = a + b
nop
pop     ebp
ret                         ; returns void; the output went
through the pointer
```

The third argument is not data but an **address**; `mov [eax], edx` dereferences it and stores the sum into the caller's variable, and the function returns `void`. The caller demonstrates the two fundamentally different ways an address is obtained, which must be read differently in disassembly:

```
lea     eax, [esp+14h]      ; &result  -> address of a LOCAL (a
stack slot)
mov     [esp+8], eax        ; pass it as the 3rd argument
...
mov     [esp+8], 407070h    ; &glob    -> address of a GLOBAL is
a constant
```

Taking the address of a **local** variable means computing a stack address at run time, which is precisely what `lea` on an `[esp+k]` or `[ebp-k]` expression does — and this is `lea` used for its literal purpose, pointer arithmetic, not as the arithmetic shortcut seen earlier. Taking the address of a **global** is trivial, because its address is a link-time constant that appears directly as an immediate (`407070h`) moved into the argument slot. An operand that is the *address of* a stack slot (via `lea`) versus an *immediate absolute address* therefore tells the analyst whether a pointer refers to local or global storage — a distinction that matters when tracing where a function's output actually lands.

9. String Instructions and the `REP` Prefix

The x86 instruction set includes a family of **string (block-memory) instructions** that move, store, compare, and scan memory using *implicit* register operands. They are compact, fast, and everywhere in compiled code and malware alike: a compiler inlines `memcpy` and `memset` as these instructions, buffer-zeroing and structure initialization appear as them, and `strlen` , `memcmp` , and search loops are built from them. Shellcode and string decryptors favor them for the same compactness. Reading them fluently is therefore essential, and the difficulty is precisely that their operands are not written out — the analyst must remember the fixed register roles.

9.1 The implicit-operand model

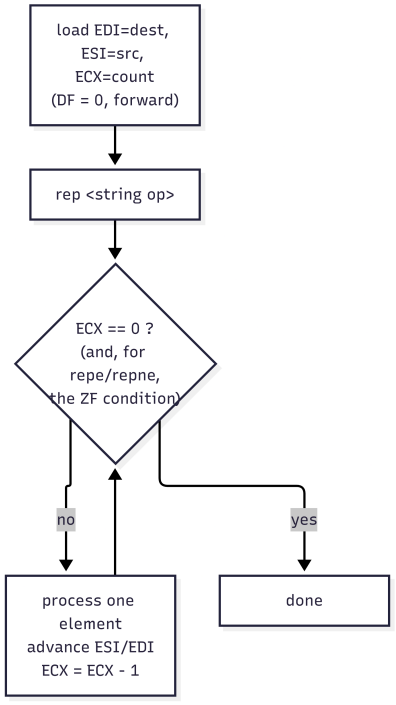
Every string instruction works on one element at a time and takes its operands from fixed registers. The source pointer is always **`ESI`** (`ds:[esi]`), the destination pointer is always **`EDI`** (`es:[edi]`), the count is always **`ECX`** , and the accumulator element is always **`AL`** / **`AX`** / **`EAX`** . A one-letter suffix sets the element size: **`b`** for a byte, **`w`** for a word (two bytes), **`d`** for a dword (four bytes). After each operation the processor **automatically advances** `ESI` and/or `EDI` by the element size. The direction of that advance is governed by the **Direction Flag** (**`DF`**) in `EFLAGS` : when `DF = 0` the pointers *increment* (forward, low to high

addresses), and when `DF = 1` they *decrement* (backward). The instructions `cld` and `std` clear and set `DF` respectively. The platform ABI keeps `DF = 0` by default, so forward processing is the norm; a `std` appearing before a string operation signals deliberate backward processing and is worth noting, as it is occasionally used to handle overlapping regions or as mild obfuscation.

A single string instruction processes just one element. To operate on a whole block it is combined with a **`REP` prefix**, which repeats it under the control of `ECX`:

- **`rep`** repeats the instruction, decrementing `ECX` each time, until `ECX = 0`. It is used with the unconditional movers `movs` and `stos` to process exactly `ECX` elements.
- **`repe`** / **`repz`** (repeat while equal) repeats while `ECX ≠ 0` **and** the zero flag `ZF = 1`. Paired with the comparators `cmps` and `scas`, it stops early the moment a *difference* appears.
- **`repne`** / **`repnz`** (repeat while not equal) repeats while `ECX ≠ 0` **and** `ZF = 0`. Paired with `cmps` and `scas`, it stops early the moment a *match* appears.

Because the register roles are fixed, the recognizable prelude to any of these routines is a short sequence that loads `EDI`, `ESI`, `ECX` (and sometimes `AL`) from the arguments, preceded by `push edi` / `push esi` to preserve those registers — `ESI` and `EDI` are callee-saved, so a function that uses them must restore them.



9.2 `MOVSB` — the `memcpy` primitive

`movsb` copies one byte from `ds:[esi]` to `es:[edi]` and advances both pointers. Prefixed with `rep` and a count in `ECX`, it copies a whole buffer — exactly `memcpy(dest, src, n)`:

```
_my_memcpy:              ; my_memcpy(dest, src, n)
push    ebp
mov     ebp, esp
push    edi
push    esi
mov     edi, [ebp+8]        ; dest  -> EDI
mov     esi, [ebp+0Ch]      ; src   -> ESI
mov     ecx, [ebp+10h]      ; n     -> ECX
rep movs byte ptr es:[edi], byte ptr ds:[esi]   ; copy n bytes
pop     esi
pop     edi
leave
ret
```

Seeing `rep movsb` should translate instantly to "copy `ECX` bytes from `ESI` to `EDI`." Its dword cousin `rep movsd` copies four bytes per step and is what a compiler typically emits for a bulk copy, often with a `rep movsd` for the dword-aligned bulk followed by a short `rep movsb` for the trailing bytes.

9.3 `STOSB` — the `memset` primitive

`stosb` stores the byte in `AL` to `es:[edi]` and advances `EDI`. With `rep` it fills `ECX` bytes with a constant — exactly `memset(dest, value, n)`:

```
_my_memset:              ; my_memset(dest, value, n)
push    ebp
mov     ebp, esp
push    edi
mov     edi, [ebp+8]        ; dest  -> EDI
mov     al,  [ebp+0Ch]      ; value -> AL (a single byte)
mov     ecx, [ebp+10h]      ; n     -> ECX
rep stos byte ptr es:[edi], al   ; write AL n times
pop     edi
```

```
leave
ret
```

The overwhelmingly common instance of this pattern is **buffer zeroing**: a compiler clears a local array or a structure with `xor eax, eax` followed by `rep stosd` (and a trailing `rep stosb`), writing zeros across the region. Recognizing `rep stos` with a zeroed accumulator as "zero-initialize this buffer" removes a great deal of noise from a function's prologue.

9.4 `CMPSB` with `REPE` — the `memcmp` primitive

`cmpsb` compares the bytes at `ds:[esi]` and `es:[edi]`, setting the flags from their difference, and advances both pointers. Prefixed with `repe`, it keeps comparing **while the bytes are equal** and `ECX` is not exhausted, stopping at the first mismatch — the engine of `memcmp` and of string equality tests:

```
_my_memcmp:              ; my_memcmp(buf1, buf2, n)
push    ebp
mov     ebp, esp
push    edi
push    esi
mov     esi, [ebp+8]        ; buf1 -> ESI
mov     edi, [ebp+0Ch]      ; buf2 -> EDI
mov     ecx, [ebp+10h]      ; n    -> ECX
repe cmps byte ptr ds:[esi], byte ptr es:[edi]   ; compare while
equal
jne     DIFF                ; loop ended on a mismatch?
mov     eax, 0              ; all bytes equal -> return 0
jmp     EXIT
DIFF:
mov     eax, 1              ; a difference was found -> return 1
EXIT:
pop     esi
pop     edi
leave
ret
```

The loop can end for two reasons, and the flags distinguish them. If every byte matched, `ECX` reached zero with `ZF = 1`, so the following `jne` is **not** taken and

the function returns `0` (equal). If a mismatch was hit, the last comparison left `ZF = 0`, the `jne` is taken, and the function returns `1` (different). The idiom "a `repe cmps` followed by a conditional jump on `ZF`" is the universal shape of a memory or string comparison; a malware analyst meets it constantly in routines that check magic bytes, passwords, or command strings.

9.5 `SCASB` with `REPNE` — the `memchr` / `strlen` primitive

`scasb` compares `AL` against the byte at `es:[edi]` and advances `EDI`. Prefixed with `repne`, it keeps scanning **while the byte does not equal `AL`** and `ECX` is not exhausted, stopping at the first match — the engine of `memchr`, byte search, and (with `AL = 0`) `strlen`:

```
_my_memchr:              ; my_memchr(buf, value, n)
push    ebp
mov     ebp, esp
push    edi
mov     edi, [ebp+8]        ; buf   -> EDI
mov     al,  [ebp+0Ch]      ; value -> AL (the byte to find)
mov     ecx, [ebp+10h]      ; n     -> ECX
repne scas al, byte ptr es:[edi]   ; scan while AL != [edi]
test    ecx, ecx            ; did the scan stop before exhausting
the count?
jnz     FOUND
mov     eax, 0              ; not found -> return NULL
jmp     EXIT
FOUND:
dec     edi                 ; EDI advanced one past the match;
step back
mov     eax, edi            ; return the address of the matching
byte
EXIT:
pop     edi
leave
ret
```

Two subtleties make this routine instructive. First, because `scasb` advances `EDI` *after* each comparison, when a match is found `EDI` already points **one byte past**

the byte that matched; the `dec edi` corrects this so the returned pointer addresses the match itself. Second, the function infers success from `ECX`: if the scan stopped with `ECX` still non-zero, it halted early on a match, so control goes to `FOUND`; if `ECX` reached zero, the whole buffer was scanned without a hit and the function returns `NULL`. The canonical application of this same instruction is `strlen`: setting `AL = 0` and scanning with `repne scasb` walks to the terminating NUL byte, and the number of bytes traversed (recoverable from `ECX`) is the string's length. Whenever `repne scasb` appears with a zeroed `AL`, it should be read as a string-length or end-of-string search.

Across all four routines the same mental substitution applies: identify the implicit registers set up before the prefixed instruction, read the prefix to learn the termination rule, and translate the whole block into its single high-level operation — copy, fill, compare, or search.

10. Anti-Analysis Techniques

Everything to this point assumed a cooperative binary: code that means what it says and can be read straight through. Malware frequently does not cooperate. It is written to waste an analyst's time, to confuse automated tools, and to hide its real instructions until the moment they execute. This final section examines a specialized instruction that often appears in such code and then two foundational evasion techniques — anti-disassembly and self-modifying code — using small routines whose true behavior is deliberately concealed.

10.1 The `LOOP` instruction

`LOOP` is a single instruction that fuses a counter decrement with a conditional branch. It **decrements `ECX`** and then **jumps to its target if `ECX` is not zero**, falling through otherwise. It is the compact realization of a counted loop, with `ECX` as the dedicated trip counter:

```
_my_loop:                ; my_loop(a, b) = a * b, by repeated
addition
push    ebp
mov     ebp, esp
mov     ecx, [ebp+8]        ; a  -> ECX (loop count)
mov     edx, [ebp+0Ch]      ; b  -> EDX (addend)
xor     eax, eax            ; result = 0
L:
```

```
add     eax, edx            ; result += b
loop    L                   ; --ECX; if (ECX != 0) goto L
leave
ret
```

The body adds `b` to an accumulator `a` times, computing `a * b`. The whole loop control is the one instruction `loop L`, equivalent to `dec ecx` / `jnz L` but a byte shorter and tied specifically to `ECX`. Recognizing `loop` as "repeat the preceding block `ECX` times" is straightforward; the reason it earns a place beside the evasion techniques is that compilers rarely emit it (they prefer `dec` / `jnz`, which is faster on modern cores), so its presence often indicates **hand-written assembly** — exactly the kind found in shellcode, decryptors, and the concealed routines below.

10.2 Anti-disassembly: position-independent code and misaligned decoding

Disassemblers reconstruct instructions by deciding where each one begins. The simplest strategy, **linear sweep**, decodes bytes straight through, assuming the byte after one instruction starts the next. Anti-disassembly defeats this assumption by arranging for the real instruction boundaries to differ from the ones a straight-through decode would pick, so that the tool renders plausible-looking but entirely wrong code.

The technique rests on two building blocks. The first is the **`call $+5` / `pop` idiom**, a way for code to learn its own address at run time — *position-independent code*, ubiquitous in shellcode that cannot rely on fixed load addresses:

```
push    ebp
mov     ebp, esp
call    $+5             ; e8 00 00 00 00  — call the very next
instruction
pop     eax             ; EAX = address pushed by the call (its
own location)
add     eax, 0Ah        ; EAX = a computed target address
jmp     eax             ; indirect jump to that address
;  ... bytes that a linear sweep now decodes as instructions ...
```

A `call` pushes the address of the following instruction as a return address; because the call target is that same following instruction, nothing is really "called" — the sole effect is to leave the current address on the stack, which `pop eax` retrieves. This is the **GetPC** idiom, and `call $+5` (opcode `E8 00 00 00 00`) is its canonical form.

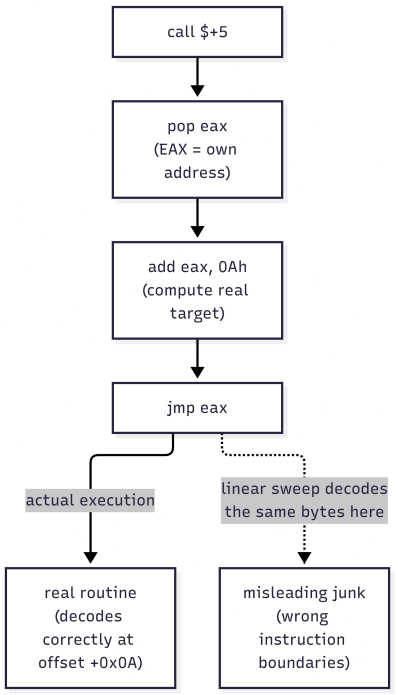
The second building block is the **indirect jump** `jmp eax`. A disassembler cannot know statically where `jmp eax` leads — the target is a run-time value — so after it, a linear sweep simply keeps decoding whatever bytes follow. Those bytes were crafted so that decoding them from the *wrong* offset yields misleading instructions, while the real control flow (`eax = own_address + 0x0A`) lands a few bytes further in, at an offset that decodes the *same* bytes into the genuine routine. The on-disk, linearly decoded view and the actually-executed view therefore diverge completely:

```
; What a linear disassembler shows after 'jmp eax' (WRONG):
inc     edx
and     eax, [edi+14BA5314h]
add     [eax], al
...                         ; plausible nonsense

; What actually executes, entered via 'jmp eax' a few bytes later
(CORRECT):
push    ebx
mov     edx, 14h            ; counter = 20
mov     ebx, 0
mov     ecx, 1
jmp     short L2
L1: mov ecx, eax
L2: lea eax, [ecx+ebx]      ; a running sum: Fibonacci step
    mov ebx, ecx
    sub edx, 1
    jne L1
pop     ebx
...
ret
```

The concealed routine is an ordinary counted loop computing a Fibonacci value — benign in itself, but the point is that its instructions were invisible to a straight

reading of the bytes. An analyst defeats this by not trusting linear output across an indirect branch: either reason about the computed target by hand (here, "own address plus ten"), instruct the disassembler to re-decode from the correct offset, or step the `jmp eax` once in a debugger to see exactly where execution goes. More capable disassemblers use **recursive-descent** decoding, which follows branch targets rather than sweeping linearly, and so resist this trick better — but an unresolved indirect jump still stops them, which is precisely why it is used.



10.3 Self-modifying code and runtime unpacking

Self-modifying code (SMC) takes concealment a step further: the real instructions are not merely hidden by misalignment, they are **not present in the file at all** in executable form. The binary stores a scrambled or encrypted blob plus a small **decoder** that rewrites that blob into genuine instructions in memory at run time, and then jumps into the now-decoded region. Static disassembly of the stored bytes yields only the decoder and a block of apparent garbage; the true payload exists only after the decoder has run. This is the governing principle of virtually every **packer** and self-decrypting malware.

Two concrete mechanisms make it work, and both are strong analytic signals. First, the decoder loop walks a range of the program's own code, transforming each unit in place:

```
call    $+5
pop     eax
add     eax, 1Eh            ; EAX = start of the region to decode
```

```
mov     edx, eax
add     edx, 24h            ; EDX = end of the region
L1:
mov     ecx, [edx]          ; read a dword of scrambled code
xchg    ch, cl              ; \
ror     ecx, 16             ;  > reverse the four bytes of the
dword
xchg    ch, cl              ; /
mov     [edx], ecx          ; write the decoded dword back over
itself
sub     edx, 4
cmp     edx, eax
jge     L1                  ; walk backward across the whole
region
```

Here the "encryption" is a trivial per-dword byte reversal (the `xchg` / `ror` / `xchg` sequence reverses the order of the four bytes), applied across a range of the function's own body; once the loop finishes, execution falls through into the region it just rewrote, which has become the same Fibonacci routine seen earlier, fronted by a small `jmp` over a junk byte. Real malware substitutes a stronger transform — an XOR key stream, RC4, or a custom cipher — but the shape is identical: a loop reading a code range, transforming it, and writing it back, followed by a transfer of control into the decoded bytes.

Second, and decisively, a program's code section is mapped **read-and-execute but not writable**, so any attempt to overwrite it would fault. Self-modifying code must therefore first make its code pages writable, and on Windows it does so with **`VirtualProtect`**, requesting the `PAGE_EXECUTE_READWRITE` protection — the constant **`0x40`**:

```
mov     [esp+8], 40h            ; flNewProtect =
PAGE_EXECUTE_READWRITE
mov     [esp],   401460h        ; lpAddress = the code to be
modified
call    _VirtualProtect@16      ; make that code region
writable+executable
```

A call to `VirtualProtect` that requests `PAGE_EXECUTE_READWRITE` (`0x40`) over a region of the program's *own code* is one of the loudest tells in static analysis: legitimate application code almost never needs to make itself writable, so its presence strongly suggests unpacking or self-modification and marks the exact spot to focus on. The practical way to recover an SMC or packed payload is dynamic: set a breakpoint just after the decode loop (or at the indirect jump into the decoded region), let the decoder run, and then read or dump the now-plaintext instructions from memory — the standard manual-unpacking workflow.

10.4 Synthesis

The evasions in this section all attack a single assumption — that the bytes on disk, read in order, are the instructions that will run. Anti-disassembly breaks the mapping between byte offsets and instruction boundaries; self-modifying code removes the real instructions from the file entirely until run time. The defenses mirror the attacks: prefer recursive-descent decoding over linear sweep, never trust a static listing across an unresolved indirect branch, treat `VirtualProtect(..., PAGE_EXECUTE_READWRITE)` over code as a red flag, and fall back to dynamic analysis — breakpoints, single-stepping, and memory dumps — whenever the static picture is deliberately false. The recurring lesson of the whole chapter reaches its sharpest form here: the instruction stream the processor actually executes is the only ground truth, and against adversarial code even the disassembly must be treated as a hypothesis to be verified at run time.

11. Analyst's Reference: Pattern-to-Construct Cues

The following cues summarize the fingerprints established in this chapter. They are stated in terms that survive stripping, since in real analysis the names are gone.

A **prologue** `push ebp` / `mov ebp, esp` (optionally followed by `sub esp, N`) marks a function entry; a matching `leave` / `ret` or `pop ebp` / `ret` marks its exit. Operands of the form `[ebp - k]` are **local variables**; operands `[ebp + 8]` , `[ebp + 0Ch]` , `[ebp + 10h]` are **incoming arguments**; operands that are **absolute addresses** are **globals**.

A `cmp` or `test` followed by a **conditional jump** is a decision point. If the jump **skips a block** and both paths reconverge, it is an **`if`** ; the tested condition is the *inverse* of the source condition. If the first block ends in an **unconditional `jmp`**

over a second block and both reconverge, it is an **`if/else`**. A **backward** conditional jump is a **loop back-edge**: if the function **jumps to the condition first** and the body is below, it is a **`while`** (or a `for`, if the body's last action is a counter increment); if the body runs **before** any test, it is a **`do/while`**.

Array access appears as an operand containing an index register scaled by the element size: `[base + reg*scale]`. An **absolute base** means a global array, an **`ebp`-relative base** means a local array, and the **scale** (1, 2, 4, 8) reveals the element size. A **`switch`** is either a **comparison ladder** — the same value compared against a run of constants, each case ending in a `jmp` to a common merge — or a **jump table** — a `cmp` / `ja` bounds check followed by an indirect `jmp [base + index*4]` into an array of code pointers in read-only data. In both, the per-case `jmp` to the merge block is the compiled `break`; its absence is fall-through.

Watch the **signedness of conditional jumps**: the signed family (`jg`, `jge`, `jl`, `jle`) indicates signed operands, while the unsigned family (`ja`, `jae`, `jb`, `jbe`) indicates unsigned operands such as sizes, lengths, counts, or pointers — often the footprint of `sizeof` or of an index-range check.

Arithmetic shortcuts recur constantly: `and reg, (2^n - 1)` is a **modulo by 2^n**; a `shl` / `shr` is a multiply/divide by a power of two; and `lea` with a bracketed expression used as a number is **linear arithmetic**, not a memory access. Hexadecimal constants should be converted on sight (`0Ah` = 10, `14h` = 20, `1Eh` = 30).

Calling convention is read mainly off the function's exit. A **plain `ret`** means the caller cleans the arguments — **`cdecl`**, the C default and the only convention that supports variadic functions; a **`ret N`** means the callee cleaned `N` bytes itself — **`stdcall`**, used throughout the Win32 API, with `N` revealing the argument footprint (`ret 0Ch` = three dwords). Arguments are read at `[ebp+8]`, `[ebp+0Ch]`, `[ebp+10h]`, and the integer result returns in `eax`. Modern compilers reserve outgoing-argument space once (`sub esp, N`) and write arguments with `mov [esp+k]` rather than `push`, folding cleanup into the epilogue, so the classic `add esp, 8` after a call may be absent or appear as a compensating `sub esp`. A result delivered by `mov [reg], ...` through a pointer argument is an **output parameter**; the pointer's origin is visible in how its address was formed — `lea` on a stack slot for a local, an immediate absolute address for a global.

String instructions carry implicit operands — source in `ESI`, destination in `EDI`, count in `ECX`, element in `AL/AX/EAX`, direction set by the `DF` flag — so they are recognized by the register setup that precedes them. A **`rep movs`** is a block copy (`memcpy`); a **`rep stos`** is a block fill (`memset`), and with a zeroed accumulator it is buffer zeroing; a **`repe cmps`** followed by a `ZF` jump is a memory/string comparison (`memcmp`); a **`repne scas`** is a byte search (`memchr`), and with `AL = 0` it is a string-length or end-of-string walk (`strlen`). The prefix states the termination rule: `rep` runs the count out, `repe/repz` stops at the first difference, `repne/repnz` stops at the first match.

Anti-analysis leaves its own fingerprints. The `loop` instruction (decrement `ECX`, jump if non-zero) is rarely compiler-generated, so it hints at hand-written assembly. The **`call $+5`** / **`pop`** idiom (`E8 00 00 00 00` followed by a `pop`) is position-independent GetPC code, the staple of shellcode. An **indirect `jmp reg`** or `call reg` with a run-time-computed target defeats linear-sweep disassembly and should never be trusted blindly — resolve it dynamically. A loop that reads a range of the program's own code, transforms each unit, and writes it back, followed by a transfer into that region, is a **decoder/unpacker**, and a **`VirtualProtect` call requesting `PAGE_EXECUTE_READWRITE` (`0x40`)** over code is the signature act of self-modifying code. All of these say the same thing: switch from static reading to dynamic analysis at that point.

Finally, and most importantly: the symbol names, inferred prototypes, and structural comments supplied by the disassembler are **hypotheses to verify, not facts**. In a genuine investigation they will be absent or deliberately misleading, and the only reliable ground truth is the instruction stream itself.

CHAPTER 2: Basic Static Analysis

Lab 03 — Basic Static Analysis

Basic static analysis is the first disciplined pass over a suspicious binary. It is performed **without ever executing the sample**: the file is treated purely as data and interrogated through its own structure — its headers, the libraries it declares, the functions it imports, and the human-readable text embedded in its sections. The objective at this stage is not to produce a definitive account of what the program does, but to generate a small set of well-grounded **hypotheses** about its capabilities. These hypotheses then direct the slower and more expensive phases of the investigation (dynamic analysis and full disassembly), telling the

analyst where to spend attention and where to ignore the overwhelming amount of irrelevant material that every Windows executable carries.

Two ideas govern everything that follows. First, **signal is scarce and noise is abundant**. A Windows program drags in a large, complex runtime, so the majority of what is visible — imports such as `CreateThread`, boilerplate from the C runtime, padding bytes — has nothing to do with the malicious behavior. The skill being built is the rapid separation of the few meaningful indicators from the mass of routine ones. Second, **nothing here is proof**. A static observation (an imported function, a string, a timestamp) is a clue whose interpretation must later be confirmed or discarded. The analyst advances by accumulating small confirmations until a coherent picture emerges.

---

1. How Executables Reuse External Code: Linking

A program interacts with the operating system through **APIs that Windows exports from its libraries (DLLs)**. By identifying precisely which functions a sample pulls from which libraries, an analyst obtains a rough but valuable map of the program's potential functionality. Understanding *how* that external code is attached to the program is therefore foundational, because the linking method determines how much of that map is visible to static analysis in the first place. There are three mechanisms.

Static linking

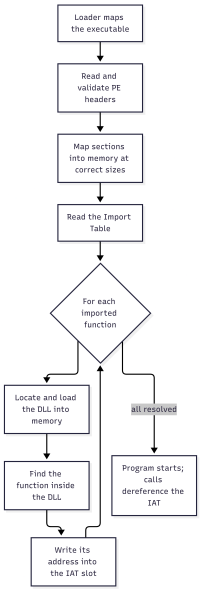
With static linking, the actual code of the library is copied into the executable **at compile time**, during the linker phase, just before the final file is written. The library's pre-compiled routines are merged with the programmer's own code and packed into a single file. Afterwards the two are indistinguishable: a disassembly shows the author's functions and the library's functions side by side, all embedded in the binary.

The advantage is self-containment — a statically linked executable has no external dependencies and will run on any compatible system even if the corresponding libraries are absent. The cost is large duplication: a widely used library is copied, in whole or in part, into every program that uses it. For this reason static linking is common in the Unix world but rare in the Windows ecosystem.

Dynamic linking

Dynamic linking is the de-facto standard on Windows. At compile time the program **declares** which functions from which libraries it needs; the linker writes the correct references into the file but does **not** embed the library code. What remains inside the executable is a set of "dangling" references — entries pointing to code that does not yet exist in the file.

Supplying that missing code is the responsibility of the **Windows loader** at execution time. When the loader maps an executable, it reads the headers, validates them, places each section in memory at the correct size, and then consults a dedicated structure in the PE header called the **import table**. That table lists every function the program uses and the library each function comes from, with an empty slot reserved for an address. The loader processes each entry in turn: it locates the library, loads it into memory if necessary, finds the target function inside it, takes the address of that function's first instruction, and writes that address into the reserved slot of the **Import Address Table (IAT)**. Once every slot is filled, the program can run: whenever the code calls a dynamically linked function, it dereferences the corresponding IAT entry and reaches the correct memory location.



The consequence for analysis is decisive. Under static linking the library **code** is visible but its identity is not; under dynamic linking the **code is absent**, but the **names of the libraries and of the imported functions are plainly listed** in the import table. Dynamic linking is thus the case that basic static analysis exploits most directly — the import list is read straight out of the file.

Runtime linking

Runtime linking is the third mechanism and the one most useful to malware authors. Here the program does **not** declare its dependencies at compile time. Instead, the code itself carries the instructions needed to locate a library at runtime, load it into memory, find the required function inside it, and call it — effectively performing the loader's job by hand, later and on demand. The canonical building blocks are `LoadLibrary` (bring a DLL into the process) and `GetProcAddress` (resolve a function's address by name within that DLL).

For legitimate software this has real benefits: a library is loaded only if and when it is actually needed, which is exactly how plugin systems work. For malware the attraction is concealment. Because the library and function names are only referenced at runtime, they need not appear in the import table at all, and the strings naming them can be obfuscated until the moment of use. A sample that relies on runtime linking can therefore interact heavily with the operating system while presenting an import table that looks almost empty — a mismatch that is itself a strong indicator.

> **Imports versus exports.** The PE format is used for both executables and libraries. An executable is expected to show many *imported* functions and essentially no *exported* ones; a library shows the opposite. A mixed file — an executable that also exports functions — is unusual and worth investigating, because it suggests the program is also meant to be loaded and called by something else.

> **Imports by name versus by ordinal.** A function can be listed in the import table either by its **name** or by its **ordinal**, the numeric position of the function inside the DLL. Importing by ordinal removes the readable name from the table, a light form of obfuscation: instead of `connect` the table shows a number. Public ordinal-to-name mappings exist for the standard system libraries, so the obfuscation is easily reversed, but it does hide the function from a careless first glance.

---

2. A Field Guide to the Common Windows Libraries

A handful of system libraries account for most of what a basic sample imports. Recognizing them on sight, and knowing what their presence implies, is the quickest route to a first hypothesis.

`kernel32.dll` contains the core operating-system functionality exposed at user level — process and thread management, file operations, memory, synchronization. It is imported by very nearly every Windows program, so its presence is unremarkable; the interest lies in *which* of its functions are used.

`advapi32.dll` provides more advanced capabilities, most importantly access to the **Windows registry** and to the **Service Control Manager**. Imports from this library point toward configuration tampering, persistence, or service installation.

`user32.dll` and `gdi32.dll` implement the graphical user interface. Malware rarely has a visible UI, so imports from these libraries deserve a second look: a sample that imports them while showing no window may be using them to inspect or manipulate *other* programs' windows — for example to read data out of them, or (via functions such as `SetWindowsHookEx`) to install a keyboard hook and build a keylogger.

`ntdll.dll` is the low-level interface to the kernel, the Windows analogue of raw system calls. Most programs reach the kernel indirectly through `kernel32.dll`, which sits above `ntdll.dll`. Microsoft never published official documentation for `ntdll.dll`; it is nonetheless stable, widely reverse-engineered, and well described in unofficial references. Direct use of it can indicate an attempt to bypass the normal, more easily monitored user-level API.

`ws2_32.dll` (and the older `wsock32.dll`) is the Winsock library, Windows' implementation of sockets for network communication. It is very close to, though not perfectly compatible with, the Unix socket interface, exposing the familiar `socket`, `connect`, `send`, `recv`, and so on. Its presence signals raw network activity. Note that opening even a single connection to a server requires six or seven distinct socket calls, so a moderately long list of Winsock imports is normal for any networked sample.

`wininet.dll` is a higher-level network library built on top of the sockets layer. It offers pre-packaged operations — download a file from a URL, issue an HTTP request — that hide the underlying connection management. Its presence suggests the sample communicates over standard web protocols (HTTP/HTTPS) rather than hand-rolled socket traffic.

`msvcrt.dll` is the Microsoft Visual C++ runtime. It is imported by essentially everything built with Microsoft's compiler toolchain and is **not** in itself an indicator of suspicious behavior; it is background, not signal.

The guiding heuristic is that **exotic imports matter more than common ones**. A `CreateProcessA` is used by countless benign programs, so it earns attention only once something more unusual has already raised suspicion. A rarely seen import — `ShellExecuteExA`, a registry or service API where none is expected — is itself grounds to dig deeper.

---

3. Reading the PE Header: What to Extract

Three pieces of header information drive the first questions in any basic analysis.

Compilation timestamp

The PE header contains a field recording the moment the executable was produced by the compiler. It is useful because it is **not** consulted during execution — the loader ignores it entirely. It exists only for the benefit of anyone inspecting the file, which also means an attacker can trivially overwrite it, and some compilers leave it meaningless. A plausible value (a real date, rather than a null or default) is therefore **a clue, not a fact**: it may genuinely record when the sample was built, but it must never be taken for granted. The file-system timestamps, by contrast, record when the file arrived on the current machine and say nothing reliable about the build.

Sections

An executable is divided into **sections**, each with a name, a size, a location, and a set of permissions. The standard ones are:

- `.text` — the executable code. This is the section that holds the program's instructions and therefore contains the **entry point**; it is marked executable (and implicitly readable) but not writable.
- `.data` — writable data used by the program at runtime. Its *virtual* size (the space it will occupy in memory) is typically much larger than its *raw* size on disk, because much of this data does not yet exist while the file sits static; the gap between the two sizes is expected and normal.
- `.rdata` — read-only data, such as constant strings and, often, import/export information. Its virtual and raw sizes are usually close.
- `.rsrc` — resources (icons, dialogs, embedded files). In a minimal program this section is small or nearly empty; a disproportionately large resource section can mean an **embedded payload** is stored there.

Two quantities repay attention. The relationship between **virtual size and raw size** characterizes a section, and a large anomaly — especially code sections with inconsistent sizes — is a red flag. **Entropy**, a measure of randomness from 0 to 8, is the other: a section packed with compressed or encrypted data shows high entropy (approaching 7.9+), whereas ordinary code and text sit well below. Unusual section **names** are equally telling; the standard `.text` / `.data` / `.rdata` / `.rsrc` set is benign, while invented names are a hallmark of packing (discussed at the end of this chapter).

Other optional-header fields

The optional header also records the **image base** (typically `0x00400000` for an executable), the **entry-point address** (expressed relative to the image base and normally located inside `.text`), and the section and file alignment values that explain the padding between sections. These confirm the file's basic sanity and locate where execution begins.

---

4. The Analysis Workflow and Tooling

The sample is **never executed** during this phase. The practical loop uses a small, free toolset:

- A **PE inspection tool** (for example PeStudio) parses the headers and presents them through a readable interface, with separate views roughly mirroring the PE structure: file header, optional header, sections, libraries, imports, and an extracted **strings** list. The compilation timestamp and the file's hashes appear on the first view.
- The hashes — typically both an MD5 and a SHA-based digest — are the file's fingerprints. Submitting a hash to an online reputation service such as **VirusTotal** reveals whether the sample has been seen before and how it has been classified, without ever uploading or running the file itself.
- The **strings** utility extracts printable character sequences from the file. It reads the file as a raw byte stream and does **not** understand the PE structure, which has two consequences: (a) imported function names, which physically live inside the import table, will also surface as strings — this is expected and duplicates the import view rather than adding to it; and (b) strings stored as **wide (UTF-16) characters** require the tool to be run in a wide-string mode, or they will be missed by a default ASCII scan.

- When an imported function is unfamiliar, its behavior is looked up in the **official Microsoft documentation**, which gives the signature, parameters, and semantics. The reading must stay shallow and purposeful: the aim is a rough sense of what a function enables, not mastery of every parameter. Detail is revisited only if later analysis proves the first pass insufficient.
- Any concrete artifact — an IP address, a domain, an unusual string — can be searched on the web to check whether it is a known **indicator of compromise (IOC)**, i.e. a data element associated with recorded malicious activity.

---

5. Worked Analysis — `Lab_03-1.exe`

The sample is a small 32-bit Windows GUI executable of 8,192 bytes. Its fingerprints are:

Property	Value
MD5	`4280aac55c1d3c327a6c00f0f0085677`
SHA-256	`84b7967aad00e982842045e7b9744af0a457d46bba70456e5f99e7eb9c`
Size	8,192 bytes
Type	PE32 executable (GUI), Intel 80386, 4 sections

5.1 When was the file compiled?

The compiler timestamp in the PE header resolves to **14 May 2009**. This is a well-formed, non-default value, consistent with a real build date rather than a compiler placeholder — and the file's Rich header confirms a **Microsoft Visual C++ 6** toolchain, which also explains the presence of `msvcrt.dll`. As always, the date is taken as indicative only, since the field is freely modifiable and plays no role in execution.

5.2 Section layout

The four sections are the standard set, which by itself argues **against** packing:

Section	Virtual size	Raw size	Entropy	Role
`.text`	3,750	4,096	5.93	Code; contains the entry point
`.rdata`	1,662	2,048	4.42	Read-only data and import information
`.data`	1,576	512	1.89	Writable runtime data (raw ≪ virtual, as expected)
`.rsrc`	124	512	3.38	Resources; nearly empty in this minimal program

The entropy values are moderate throughout, the names are conventional, and the entry point lies within `.text`. Nothing in the section table suggests obfuscation; the code is available for straightforward inspection.

5.3 Imports and the hypotheses they generate

The sample imports from **five** libraries. Four of them — `kernel32.dll`, `user32.dll`, `shell32.dll`, and `msvcrt.dll` — are immediately recognizable, and the fifth, `ws2_32.dll`, is the Winsock library. The distribution of imported functions is itself informative: a single function from `user32.dll`, only two from `shell32.dll`, a larger set from Winsock, and the bulk from `kernel32.dll`.

The functions that matter group into four capabilities.

Network communication (`ws2_32.dll`). The sample imports `WSAStartup`, `socket`, `connect`, `send`, `recv`, `closesocket`, `htons`, and `inet_addr`. This is precisely the minimal set needed to initialise Winsock, create a socket, resolve and connect to a hard-coded address, and exchange data — the signature of a program that **opens an outbound network connection**. Notably, every one of these functions is **imported by ordinal rather than by name** (ordinals 115, 23, 4, 19, 16, 3, 9, 11): in a raw PE view they appear as bare numbers, a mild obfuscation that a PE tool or a public ordinal table resolves back to the familiar socket calls.

Shell interaction (`shell32.dll`). The two imports are `ShellExecuteExA` and `SHChangeNotify`. The first is the significant one: `ShellExecuteExA` launches other programs through the shell and is rare enough in ordinary software that its mere presence justifies closer study (see 5.4). This is a far stronger indicator than the process/thread functions below, which are common even in benign code.

Process, thread, and pipe management (`kernel32.dll`). The sample imports `CreateProcessA`, `CreateThread`, `TerminateThread`, `TerminateProcess`, together with `CreatePipe`, `PeekNamedPipe`, `DisconnectNamedPipe`, `DuplicateHandle`, and the file primitives `ReadFile` and `WriteFile`. Process and thread creation are unremarkable in isolation, but **in combination with pipes and with an outbound socket** they form a recognizable pattern: a child process whose standard input and output are redirected through pipes and relayed over the network — in other words, a **remote shell**. `GetComputerNameA` additionally lets the program identify the host it is running on.

Host/environment helpers. `GetEnvironmentVariableA`, `GetShortPathNameA`, `GetModuleFileNameA`, and the string helpers `lstrcatA` / `lstrcpyA` let the program read its own location and build command strings — relevant to the self-deletion behavior discussed below.

The single `user32.dll` import, `LoadStringA`, is minor (it reads a string resource) and does not change the picture.

5.4 Deep dive: `ShellExecuteExA` and `SHELLEXECUTEINFO`

`ShellExecuteExA` is the standout import, and its documentation illustrates a recurring Windows pattern. The function itself takes a **single parameter**: a pointer to a `SHELLEXECUTEINFO` structure. Early Windows APIs accumulated so many parameters over successive versions that Microsoft eventually collapsed them into one structure with many fields — so the one-line description of the function is uninformative, and the real meaning lives in the structure's fields.

Two fields carry the payload of intent. **`lpVerb`** is a string naming the action to perform — among the allowed verbs are `open`, `edit`, `explore`, `print`, `properties`, and `runas` (execute with elevated privileges). **`lpFile`** names the file or object the verb acts upon, and **`lpParameters`** supplies optional command-line arguments to pass to it. The structure also contains an **`HWND` handle**; a *handle* in Windows is an opaque reference to an operating-system object — a file, a network connection, a window, a UI control — analogous to a Unix file descriptor but used far more pervasively.

The takeaway is that `ShellExecuteExA` can launch an arbitrary executable with arbitrary arguments and an arbitrary verb. The exact program and arguments are not fixed by the function; they are assembled by the surrounding code and would

be recovered, during later disassembly, by reading the instructions that populate the `SHELLEXECUTEINFO` fields immediately before the call.

5.5 Strings that stand out

Filtering out the noise — short fragments such as `D$`, `QJ`, and the long `PADDING` run — several strings are meaningful, and together they confirm and extend the import-based hypotheses.

- **`60.248.52.95:443`** — an **IP address with a port**, stored as a **wide (UTF-16) string**, which is why it surfaces only under a wide-character scan. This both reinforces the network hypothesis (the Winsock imports now have a concrete destination) and supplies a hard indicator. The port **443** conventionally carries HTTPS, suggesting the sample may attempt a TLS-encrypted channel to this address. Public reputation lookups flag this IP as a known indicator of compromise associated with historical malware analysis material.
- **`http://www.ueopen.com/test.html`** — an embedded URL, physically preceded by a stray `0x01` byte. A reachable remote resource of this kind is a second network artifact; the leading non-printable byte hints that the string is consumed by code rather than displayed.
- **`cmd.exe`**, **`COMSPEC`**, **`/c del`**, and **`> nul`** — the components of a **command-shell invocation**. `cmd.exe` is the Windows command interpreter; `COMSPEC` is the environment variable that points to it (so the path to the interpreter need not be hard-coded); `/c` runs a single command and exits; `del` deletes a file; and `> nul` discards the command's output (the Windows equivalent of redirecting to `/dev/null`). The strings `cmd.exe` and `/c del` appear *separately* because they are copied into two different fields of the `SHELLEXECUTEINFO` structure — the executable in `lpFile`, the arguments in `lpParameters`. The file actually being deleted is not present among the strings, which implies it is **determined at runtime** — most plausibly the program's own path, obtained via `GetModuleFileNameA` and appended to the `del` command.
- **`urlmon.dll`** — named in the strings but **absent from the import table**. This mismatch is the fingerprint of a library **loaded at runtime** (via `LoadLibrary` / `GetProcAddress`) rather than at load time. `urlmon.dll` provides URL and Internet-protocol handling (notably file downloads), a third confirmation of network capability whose exact use can only be pinned down later, in the code.

- `ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789-/` — a 64-character alphabet. A string of this exact shape is the lookup table for a **Base64-style encoding** (here a variant using `-` in place of the usual `+`). Its presence indicates the sample encodes or lightly obfuscates data — most likely its network traffic — rather than sending it in the clear.
- `*(SY)#`, `*(SY)# cmd`, and `send = %d` — operational strings. `*(SY)#` has the form of a **command prompt** that a backdoor would send to its operator, and the `cmd` suffix ties it to launching the shell. The `%d` placeholder marks a format string (used with `sprintf`-style functions) assembled at runtime, consistent with reporting the number of bytes sent.

5.6 What happens when the sample runs

Executed in a protected environment (and only after a backup copy is made), the program appears to do nothing visible — the common experience with malware — but one observable effect does occur: **the executable deletes itself**. This is exactly the behavior the `cmd.exe` + `/c del` + `COMSPEC` strings predicted, and it explains why the target filename was absent from the static strings: the program inspects its **own** path at runtime and appends it to the delete command.

Self-deletion is a classic concealment technique. The sample is designed to be loaded into memory, remain resident performing its function, and simultaneously remove its on-disk copy, leaving a **memory-only** process that leaves no file behind for the user or a casual examiner to find. (A fuller account of the runtime behavior — whether the network connection is actually established, and what traverses it — belongs to dynamic analysis, which observes the running process directly.)

5.7 Host-based and network-based signatures

Even at the static level the sample yields concrete indicators.

Host-based signatures. The reference to `cmd.exe` (shell execution on the host) and the runtime-loaded `urlmon.dll` are host-level artifacts. The self-deletion is itself a host-based behavioral signature. Registry- and process-level artifacts would be enumerated during dynamic analysis.

Network-based signatures. The hard-coded endpoint `60.248.52.95:443`, the embedded URL `http://www.ueopen.com/test.html`, and the use of port **443** are the network indicators. Together they define what a defender would watch for on the wire.

Indicator	Type	Significance
`60.248.52.95:443`	Network (IP + port)	Hard-coded C2 endpoint; known IOC; port 443 implies TLS
`http://www.ueopen.com/test.html`	Network (URL)	Embedded remote resource
`cmd.exe` , `COMSPEC` , `/c del`	Host (shell command)	Shell execution and self-deletion
`urlmon.dll`	Host (runtime-loaded DLL)	Network capability resolved at runtime
MD5 `4280aac5…` / SHA-256 `84b79…`	File hash	Sample identity for reputation lookup

5.8 What impeded the analysis, and how to overcome it

The principal limitation of basic static analysis is reached here directly: several questions cannot be answered from the file alone. The exact **filename deleted at runtime** is not in the strings because it is built dynamically; whether the **network connection is actually opened** and what data crosses it cannot be confirmed without running the sample; and the role of the runtime-loaded **`urlmon.dll`** is only inferable. These gaps are overcome by the next two phases — **dynamic analysis** (observing the process, its file and registry activity, and its network traffic as it runs, for example with a process monitor filtered to the sample and its children) and **full disassembly** (reading the code that assembles the `SHELLEXECUTEINFO` structure and resolves the runtime imports). Notably, the self-deletion did **not** obstruct the investigation: because the standard discipline is never to run the sample during static analysis, the only on-disk copy was preserved, and the deletion instead became an informative observation rather than a loss of the specimen.

5.9 Purpose of the sample

Assembling every confirmed clue — outbound Winsock connection to a hard-coded `IP:443` , `cmd.exe` launched through `ShellExecuteExA` , standard I/O marshalled through pipes, host identification via `GetComputerNameA` , a Base64-style encoding table, a command-prompt string `*(SY)#` , runtime-loaded

`urlmon.dll`, and self-deletion for stealth — the sample is a **network backdoor / remote shell**. It connects to a remote controller, hands the operator an interactive command shell on the infected host with its traffic lightly encoded, and erases its own file to survive only in memory. Every element of this conclusion is a hypothesis grounded in a static observation and would be confirmed, one piece at a time, by dynamic analysis and disassembly.

```
Error parsing Mermaid diagram!

Lexical error on line 10. Unrecognized text.
...untime-loaded urlmon.dll .-> URL[http://
-----------------------^
```

---

6. Bridge: From Clean Samples to Packed Samples

`Lab_03-1.exe` was cooperative: standard sections, a readable import table, and informative strings. Not every sample is. **Packing** is a common technique for defeating exactly the basic static analysis performed above. A packed program has had its real code **compressed or encrypted** and wrapped inside a small stub containing only the few routines needed to **load the packed body into memory, restore it, and transfer execution to it**. From the outside, a packed sample reveals almost nothing: its import table shrinks to a handful of functions (often just `LoadLibrary`, `GetProcAddress`, and memory-allocation calls), its strings dissolve into high-entropy garbage, and its sections carry unusual names and near-maximal entropy. Detecting these tell-tales — anomalous section names, high entropy, a suspiciously small import table — and deciding whether the sample can be unpacked is the subject of the next samples in this lab.

Lab 03 — Basic Static Analysis (Part 2: Samples 2–4)

The first sample was deliberately cooperative — standard sections, a readable import table, informative strings. The remaining three raise the difficulty in two directions: the second introduces a different command-and-control model (HTTP rather than raw sockets, with active camouflage), while the third and fourth introduce **packing**, the technique whose whole purpose is to defeat the basic static analysis performed so far. Each is examined with the same disciplined loop: hashes

and reputation, section layout, imports, strings, and the hypotheses they license — never executing the sample.

---

1. `Lab_03-2.exe` — The HTTP Backdoor

An 8,704-byte 32-bit GUI executable. Its fingerprints are:

Property	Value
MD5	`02658bc9801f98dfdf167accf57f6a36`
SHA-256	`8a35842d3f5963f715def0bbd0a53d7ffaae2d2ca79f56a5ac8bede647`
Size	8,704 bytes
Type	PE32 executable (GUI), Intel 80386, 4 sections

1.1 Reputation and compilation

Submitting the MD5 to a reputation service identifies the file as previously seen, classified across engines as a backdoor/downloader. The compiler timestamp resolves to **16 September 2008**; as always, it is treated as indicative rather than authoritative. The section layout is the standard, unremarkable `.text` / `.rdata` / `.data` / `.rsrc` set with moderate entropy — the sample is **not packed**, and its code and strings are available for direct inspection.

1.2 Imports and hypotheses

The import table is dominated by a single, decisive library.

`wininet.dll` — high-level web communication. The sample imports the full WinINet request chain: `InternetOpenA`, `InternetConnectA`, `HttpOpenRequestA`, `HttpSendRequestA` / `HttpSendRequestExA` / `HttpEndRequestA`, `InternetReadFile`, `InternetWriteFile`, `InternetQueryOptionA` / `InternetSetOptionA`, and `InternetCloseHandle`. This is the complete toolkit for driving an HTTP(S) conversation — opening a session, connecting to a server, issuing requests, and streaming data in both directions. Where the first sample built its channel from raw Winsock primitives, this one communicates over **standard web protocols**, which blend into ordinary traffic far more easily.

`kernel32.dll` — shell plumbing and files. `CreateProcessA` together with `CreatePipe` and `PeekNamedPipe`, and `ReadFile` / `WriteFile`, reproduce the familiar pattern of a child process whose input and output are marshalled through pipes — here almost certainly `cmd.exe`, so that commands received over HTTP can be executed and their output captured. `GetWindowsDirectoryA` locates system paths for dropping or staging files.

`msvcrt.dll` — runtime and file I/O. Beyond the usual runtime boilerplate, the C runtime's file functions (`fopen`, `fread`, `fwrite`, `fseek`, `ftell`, `fclose`) indicate the program **reads and writes files on disk** — consistent with a download/upload capability.

The combined hypothesis from imports alone is a **remote-access tool that speaks HTTP, executes shell commands, and transfers files**.

1.3 Strings that stand out

The strings confirm and sharpen every part of that hypothesis, and reveal the sample's camouflage.

- **`69.25.50.10`** — a hard-coded **command-and-control (C2) IP address**, the destination the WinINet calls will target. A concrete network IOC.
- **`wuauclt.exe`** — the filename of the legitimate **Windows Update Automatic Update Client**. Its presence is a strong sign of **masquerading**: the backdoor impersonates (or launches itself under the identity of) a trusted system process so that its network activity looks like routine Windows Update traffic.
- **`Hello.I am here!`** — a **beacon / check-in message**, the announcement a newly active implant sends to its controller to signal that the host is compromised and awaiting instructions.
- **`Begin Download`**, **`Begin Upload`**, **`getf`**, **`putf`**, **`type`** — the backdoor's **command vocabulary**. `getf` / `putf` download and upload files; `type` prints a file; the "Begin Download/Upload" markers frame the transfers. The usage string **`use error! putf [transpeed] [filepath]`** is an operator help message that confirms `putf` takes a transfer-speed and a file path.
- **`cmd /c`** — the prefix for **running an arbitrary command** through the interpreter, tying the `CreateProcessA`/pipe imports to remote command execution.
- **`\tasks\`** — a directory used to stage **tasks** fetched from the controller.

- `CONNECT` and `HTTP/1.0` — the sample supports the HTTP `CONNECT` method, i.e. it can tunnel through an **HTTP proxy**, allowing it to reach its C2 from inside networks that only permit proxied web access.
- `<head>` / `</head>` — the implant **parses HTML** returned by the server, a common way to smuggle commands inside pages that look benign to a casual observer.
- `Type command disable.Go on!` and `begin` — further protocol/status strings exchanged with the controller.

1.4 What happens when it runs, and its signatures

Executed in isolation the program produces no visible window; its activity is entirely network- and file-oriented. Behaviorally it initialises a WinINet session, beacons to `69.25.50.10` with `Hello.I am here!`, and then waits for commands — downloading files, uploading files, or executing shell commands via `cmd /c` and returning their output — while presenting itself as `wuauclt.exe`.

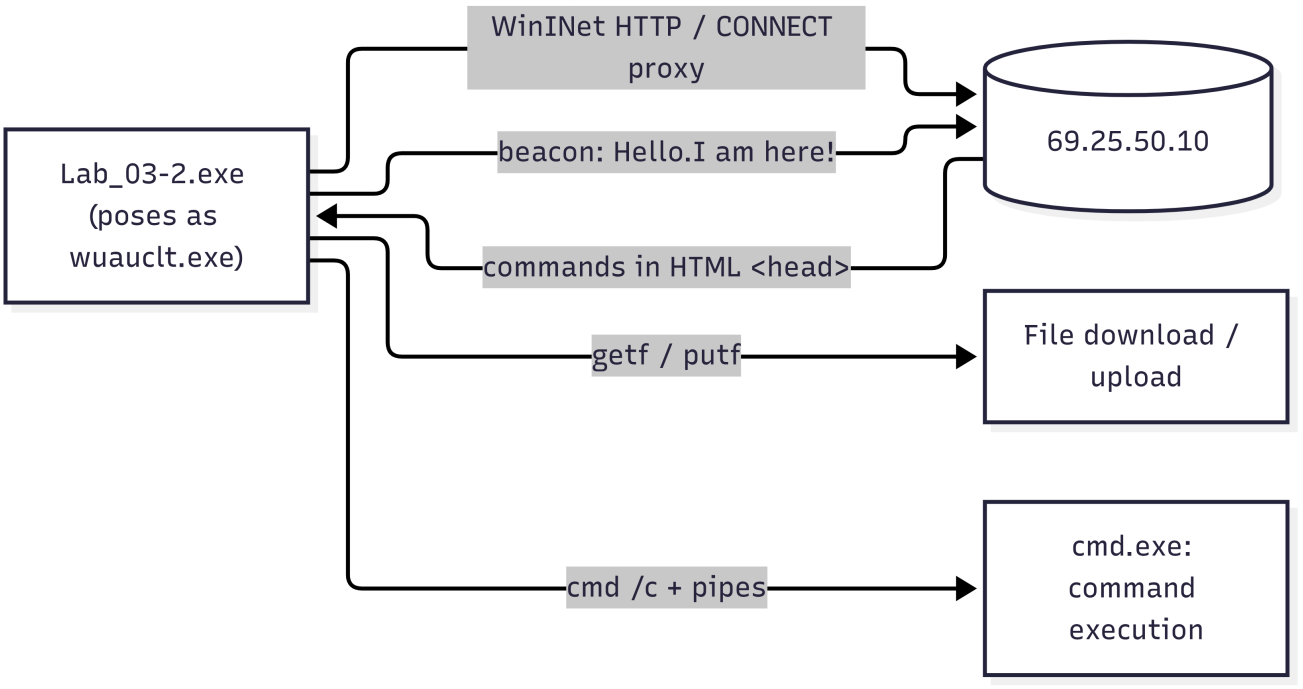
Host-based signatures. The impersonation of `wuauclt.exe`, the `\tasks\` staging directory, the spawning of `cmd.exe`, and files written or read on disk are the host-level indicators. **Network-based signatures** are the C2 address `69.25.50.10`, the beacon string `Hello.I am here!`, the HTTP request structure (including the `CONNECT`/proxy behavior), and the `getf` / `putf` transfer markers on the wire.

On the dynamic questions. Confirming *which* URL path is requested, what the beacon interval is, and whether a proxy is actually used requires running the sample under observation — a process monitor filtered to the sample's process name (and its `cmd.exe` children), plus a network capture. A sensible process-monitor filter isolates events whose *Process Name* is the sample (or `wuauclt.exe`) to cut away the enormous volume of unrelated system activity; a network-focused filter narrows to connections toward `69.25.50.10`. These belong to the dynamic-analysis phase.

1.5 Purpose

`Lab_03-2.exe` is an **HTTP backdoor / remote-access tool**. It beacons to a hard-coded controller over HTTP (tunnelling through a proxy when necessary), disguises its traffic as Windows Update activity by posing as `wuauclt.exe`, and gives its operator file download/upload and arbitrary command execution on the infected host. It is functionally close to the first sample — a remote shell with file

transfer — but trades raw sockets for the stealthier, proxy-friendly web-protocol channel.



---

2. `Lab_03-3.exe` — A Packed Sample (Custom Packer)

A 14,336-byte 32-bit **console** executable. Its fingerprints are:

Property	Value
MD5	`6c2a33512b8b0ec906982783e82b9678`
SHA-256	`212b29a8e36ccc8f65205122e33d84940a156a9a91329f747a713f988a`
Size	14,336 bytes
Type	PE32 executable (console), Intel 80386, 3 sections

2.1 Is it packed? Packing recognised from the header

Every structural indicator points to **packing**, and together they are conclusive.

Anomalous section names. The sections are named `LOL0`, `LOL1`, and `.rsrc` — the first two are invented names, not the conventional `.text` / `.data` / `.rdata`. Non-standard section names are a hallmark of a packer, which carves the file into its own layout.

A section with virtual size but no raw data. `LOL0` has a virtual size of 28,672 bytes but a **raw (on-disk) size of 0**. An empty-on-disk, large-in-memory section is the **destination** into which the stub will write the unpacked code at runtime: the space is reserved in memory, but there is nothing to store on disk because the real code does not yet exist in cleartext.

A high-entropy data section. `LOL1` carries 12,800 bytes of raw data at **entropy 7.82** (out of a maximum of 8). Near-maximal entropy means the content is **compressed or encrypted** — this is the packed body of the real program.

Executable *and* writable sections. Both `LOL0` and `LOL1` are marked as simultaneously writable and executable. Legitimate code sections are executable but not writable; a section that is both is a region the program intends to **rewrite and then run**, exactly what an unpacking stub does.

A minimal, revealing import table. Only a handful of functions are imported: `LoadLibraryA`, `GetProcAddress`, `VirtualAlloc`, `VirtualProtect`, `VirtualFree`, and `ExitProcess`. This is precisely the **unpacking stub's** toolkit — allocate memory (`VirtualAlloc`), write the decompressed code into it and make it executable (`VirtualProtect`), then resolve the real APIs at runtime (`LoadLibraryA` / `GetProcAddress`). The genuine imports of the underlying program are hidden inside the packed body and will only materialise after unpacking. The two further imports that *do* leak — `InternetOpenA` (WinINet) and `WSAStartup` (Winsock) — are a tantalising hint that the hidden payload has **network capability**.

The sample is therefore packed, but the packer is **not a standard, recognised one**: the section names `LOL0` / `LOL1` are a bespoke marker rather than the signature of a known tool such as UPX.

2.2 Can it be unpacked?

Not with an automated, format-aware unpacker. A UPX unpacker, for instance, refuses the file outright — it is not UPX, and its sections do not match any tool UPX understands. Because the packer is custom, there is no off-the-shelf utility that recognises its format and reverses it. Unpacking it requires the **manual** approach: run the sample under a debugger in a protected environment, let the stub decompress the real code into memory, stop execution at the **original entry point** (the moment control transfers from the stub to the unpacked program), and **dump** the reconstructed process image from memory — rebuilding its import

table afterward. This crosses into dynamic and advanced techniques and lies beyond basic static analysis.

2.3 Strings, and what they tell us

The direct consequence of packing is visible in the strings: there are **almost none that are meaningful**. The readable text dissolves into high-entropy fragments, which is itself the strongest confirmation that the payload is compressed or encrypted. A few artifacts leak from the stub and the resource section — the library names `KERNEL32.DLL`, `WININET.dll`, `WS2_32.dll`; the stub API names; a `http://%s` format fragment and a `Mozilla`-style user-agent remnant suggesting the hidden payload builds HTTP requests; and a short token `KIZXORXZWV` containing the substring `XOR`, hinting at an XOR-based decoding step. The scarcity of strings is the finding: a program whose strings have vanished is advertising that it is packed.

2.4 Behaviour, signatures, and purpose

With the real code encrypted, basic static analysis can state only what the structure implies: the leaked `InternetOpenA` and `WSAStartup`, together with the `http://%s` and user-agent fragments, indicate that the unpacked program performs **network communication** — most plausibly another downloader or backdoor contacting a remote host. No concrete host- or network-based signatures (specific IPs, domains, or filenames) can be extracted at this level, because they are inside the packed body. The thing that **impeded the analysis is the packing itself**, and the way to overcome it is to unpack the sample (manually, as above) and then repeat the ordinary static loop on the recovered code. The purpose can only be hypothesised as a **packed network backdoor/downloader**; a definitive answer awaits unpacking.

---

3. `Lab_03-4.exe` — A UPX-Packed Service-Installing Backdoor

A 34,816-byte 32-bit GUI executable. Its fingerprints are:

Property	Value
MD5	`9f880ac607cbd7cdfffa609c5883c708`

Property	Value
SHA-256	`a0dbee0e7238d028e4efd54fa24b94dc10c6a5f488b09ba49b37d90f22`
Size	34,816 bytes (packed)
Type	PE32 executable (GUI), Intel 80386, **UPX compressed**, 3 sections

3.1 Is it packed, and with what?

Yes, and here the packer is a **known** one. The sections are named `UPX0`, `UPX1`, and `.rsrc` — the unmistakable signature of **UPX (the Ultimate Packer for eXecutables)**. As with the previous sample, `UPX0` has a large virtual size (77,824 bytes) but **zero raw data** (the unpack destination), while `UPX1` holds the compressed body at **entropy 7.94** (essentially maximal). The import table of the packed file is correspondingly minimal — `LoadLibraryA`, `GetProcAddress`, `ExitProcess`, and a bare handful more — the UPX stub's toolkit.

3.2 Can it be unpacked? Yes

Because the packer is standard, the sample unpacks cleanly and automatically. UPX stores enough metadata to reverse its own compression, so running the unpacker in decompress mode restores the original file — here expanding it from 34,816 bytes back to **110,592 bytes**, with the normal `.text` / `.rdata` / `.data` / `.rsrc` sections, moderate entropy, and a **full, readable import table and string set** recovered. This is the decisive difference from `Lab_03-3.exe`: a known packer is trivially reversed, whereas a custom one is not. All of the analysis below is performed on the **unpacked** image.

3.3 Imports and strings after unpacking

The recovered file is far richer than the packed stub suggested and reveals a two-stage design: a **dropper** that installs a persistent **service**.

Dropper / resource extraction (`kernel32.dll`). `FindResourceA`, `SizeofResource`, `LoadResource`, `LockResource`, and `FreeResource` read an embedded **resource**, which `CreateFileA` / `WriteFile` write to disk. The resource stored in the file is **not a plain executable** — its first bytes are not the `MZ` signature but encoded data — so the dropper **decodes the resource at runtime** before writing it out as the payload DLL. `GetFileTime` / `SetFileTime` then **timestomp** the dropped file, copying an existing file's timestamps onto it so the new file does not stand out as recently created.

`GetSystemDirectoryA` / `GetWindowsDirectoryA` and `lstrcatA` build the destination path in the system directory.

Service installation and persistence (`advapi32.dll`). `OpenSCManagerA`, `CreateServiceA`, `StartServiceA`, and the registry functions `RegCreateKeyA` / `RegSetValueExA` / `RegOpenKeyExA` install the payload as a **Windows service**. The strings make the mechanism explicit:

- **`Rasmon.dll`** and **`\Rasmon.dll`** — the name of the dropped payload DLL.
- **`ServiceMain`** and **`ServiceDll`** — the DLL exports a `ServiceMain` entry point and is registered as the `ServiceDll` value under the service's **`Parameters`** key.
- **`%SystemRoot%\System32\svchost.exe -k SysIns`** and **`... -k netsvcs`** — the service is configured to run **inside `svchost.exe`** as part of a service group (`SysIns`, added alongside the standard `netsvcs`). Hosting a malicious DLL inside the trusted `svchost.exe` is a classic persistence-and-camouflage technique.
- **`SYSTEM\CurrentControlSet\Services\%s\Parameters`**, **`SYSTEM\CurrentControlSet\Services\%s\Security`**, and **`SOFTWARE\Microsoft\Windows NT\CurrentVersion\SvcHost`** — the exact registry paths written to register the service, its parameters, and its `svchost` group membership.

Capabilities revealed in the recovered strings. The names surfaced after unpacking describe the payload's intended reach:

- **Privilege escalation** — `AdjustTokenPrivileges`, `LookupPrivilegeValueA`, and the privilege names **`SeDebugPrivilege`** and `SeShutdownPrivilege`, used to gain debug rights over other processes.
- **Anti-forensics** — **`ClearEventLogA`**, `OpenEventLogA`, `CloseEventLog`: the ability to **wipe the Windows event logs** and erase traces of the intrusion.
- **System and process reconnaissance** — `CreateToolhelp32Snapshot` (enumerate running processes), `EnumServicesStatusA` (enumerate services), `GetLogicalDrives` / `GetDriveTypeA` / `GetDiskFreeSpaceA` (enumerate drives), `FindFirstFileA` / `FindNextFileA` (traverse the file system), and reads of `HARDWARE\DESCRIPTION\System\CentralProcessor\0` (fingerprint the CPU).
- **Associated components** — `VedioDriver.dll` (note the characteristic misspelling of "Video"), `acelpvc.dll`, `mdm.exe`, and

`\drivers\etc\networks.ics`: additional modules the payload references, consistent with a remote-control / screen-capture capability installed alongside the service.

- **Self-deletion** — a dropped batch file (`%s\DFS.bat`) built around `if exist "%s" goto pp` / `del "%s"` loops until it can delete the original dropper, removing the installer after the service is in place.

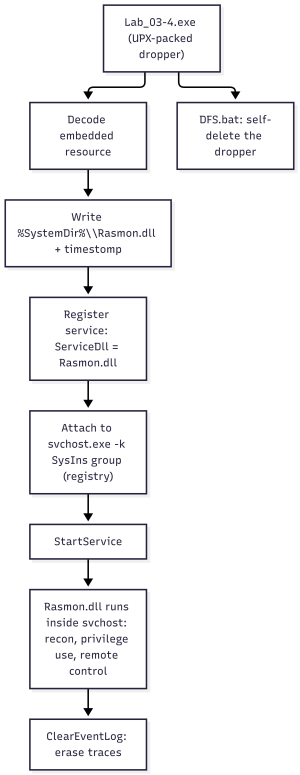
3.4 Behaviour and signatures

On execution the dropper decodes its embedded resource, writes it to the system directory as **`Rasmon.dll`**, timestomps it, registers it as a `svchost`-hosted service (adding the `SysIns` group), starts the service, and deletes itself via `DFS.bat`. From that point the malicious code runs as a legitimate-looking system service under `svchost.exe`.

Host-based signatures: the dropped file `%SystemDir%\Rasmon.dll` (and the associated `VedioDriver.dll`, `acelpvc.dll`, `mdm.exe`); the new service and its registry footprint under `SYSTEM\CurrentControlSet\Services\<name>` with `ServiceDll = ...\Rasmon.dll` and membership in the `SysIns` `svchost` group under `...\SvcHost`; the `DFS.bat` self-deletion artifact; and cleared event logs. **Network-based signatures** are not visible in this dropper stage — the remote-communication logic lives inside the dropped `Rasmon.dll` and would be analysed by extracting and examining that DLL separately.

3.5 Persistence strategy

The sample achieves persistence by **installing itself as a Windows service whose logic is a DLL (`Rasmon.dll`) hosted inside `svchost.exe`**. Registering the payload as a `ServiceDll` under a service's `Parameters` key and attaching it to a `svchost` service group means the operating system **launches the malware automatically at every boot**, as part of a trusted system process, with no file of its own left running in the open. Combined with timestomping, event-log clearing, and self-deletion of the dropper, this makes the implant both durable across reboots and difficult to notice.



---

4. Synthesis Across the Four Samples

The lab's four specimens form a deliberate progression. The first two are **unpacked backdoors** that differ only in their C2 channel — raw Winsock to `60.248.52.95:443` with a lightly encoded shell (sample 1), versus HTTP/WinINet to `69.25.50.10` disguised as `wuauclt.exe` with file transfer and proxy support (sample 2). The last two introduce **packing** as an anti-analysis layer and teach the central lesson of basic static analysis of obfuscated code: recognise packing from its structural fingerprints (anomalous section names, a zero-raw/large-virtual section, near-maximal entropy, executable-and-writable sections, a minimal stub import table), then judge whether the packer is **standard** (UPX — reversed automatically, exposing the full service-installing dropper of sample 4) or **custom** (the `LOL0` / `LOL1` packer of sample 3 — requiring manual unpacking in a debugger). In every case the method is the same: extract what the file's own structure is willing to reveal, form grounded hypotheses, and carry the unanswered questions forward to dynamic analysis and disassembly.

Basic Static Analysis — Exam Quick-Reference Sheet

A scanning aid for the exam: what to look at, in what order, and how to read it. The first half is sample-independent reference (imports → capabilities, packing tells, PE fields, string categories, workflow). The second half is the consolidated IOC tables for the four Lab 03 samples.

---

1. The Workflow, in Order

Step	Question it answers	Where to look
Hashes (MD5 / SHA-256)	Has this been seen before?	PE tool first view → VirusTotal
Compilation timestamp	When was it built?	PE / File header (**treat as a clue, not a fact**)
Sections	Is it packed? What is where?	Sections table (names, raw vs virtual size, entropy)
Libraries + imports	What *can* it do?	Imports/Libraries view → form hypotheses
Strings (ASCII **and** wide)	IPs, URLs, commands, filenames	Strings view → web-search the artifacts
(Unpack if needed)	Reveal hidden code	Automatic for UPX; manual/debugger for custom

> **Golden rules:** never execute the sample during static analysis · signal is rare, noise is huge · every finding is a hypothesis to confirm later · exotic imports matter more than common ones · always run strings in **wide/UTF-16** mode too.

---

2. Imports → Likely Capability

Read the import list as a capability map. The high-value functions to flag:

Import(s)	Library	Suggests
`socket`, `connect`, `send`, `recv`, `WSAStartup`, `inet_addr`, `htons`	ws2_32	Raw socket network comms (backdoor / reverse shell)
`InternetOpenA`, `InternetConnectA`, `HttpOpenRequestA`, `HttpSendRequestA`, `InternetReadFile`	wininet	HTTP(S) C2 / downloader
`URLDownloadToFile`	urlmon	Download a file from a URL
`CreateProcessA` + `CreatePipe` + `PeekNamedPipe` + `ReadFile` / `WriteFile`	kernel32	Shell with redirected I/O (remote shell)
`ShellExecuteExA` / `ShellExecuteA` (+ `SHELLEXECUTEINFO`)	shell32	Launch another program/command (**rare → suspicious**)
`cmd.exe` / `COMSPEC` strings + `/c`	—	Run a shell command; `/c del` = self-deletion
`LoadLibraryA` + `GetProcAddress`	kernel32	**Runtime linking** (hides real imports)
`VirtualAlloc` + `VirtualProtect` + `VirtualFree`	kernel32	Allocate+rwx memory → unpacking / injection
`OpenSCManagerA`, `CreateServiceA`, `StartServiceA`	advapi32	Install a **service** (persistence)
`RegCreateKeyA`, `RegSetValueExA`, `RegOpenKeyExA`	advapi32	Registry read/write (config, persistence)
`FindResourceA`, `LoadResource`, `LockResource`, `SizeofResource`	kernel32	Read an **embedded payload** from `.rsrc` (dropper)
`GetFileTime` + `SetFileTime`	kernel32	**Timestomping** (anti-forensics)

Import(s)	Library	Suggests
`AdjustTokenPrivileges`, `LookupPrivilegeValueA` (+ `SeDebugPrivilege`)	advapi32	Privilege escalation / debug rights
`ClearEventLogA`, `OpenEventLogA`	advapi32	**Wipe event logs** (anti-forensics)
`CreateToolhelp32Snapshot`, `EnumServicesStatusA`, `GetLogicalDrives`	kernel32/advapi32	Host/process/drive reconnaissance
`SetWindowsHookEx`	user32	Keyboard/input hook → **keylogger**
`FindFirstFileA` / `FindNextFileA`	kernel32	File-system traversal
`GetComputerNameA`	kernel32	Identify the victim host

Library at a glance: kernel32 = core OS (ubiquitous) · advapi32 = registry + services · user32/gdi32 = GUI (rare in malware → look twice) · ntdll = low-level/undocumented kernel · ws2_32/wsock32 = sockets · wininet = HTTP · msvcrt = C runtime (**noise, not a signal**).

Context rule: `CreateProcessA`, `CreateThread`, `ReadFile` etc. are common and benign *alone* — treat them as meaningful only once a stronger indicator (an exotic import, an IP, a packer) is already present.

---

3. Linking — Why Some Imports Are Invisible

Type	Resolved	Code visible?	Names visible?	Re
Static	Compile time	Yes (embedded)	No	Ra Wi
Dynamic	Load time (loader → IAT)	No	**Yes** (import table)	Th nor cas sta ana rea

Type	Resolved	Code visible?	Names visible?	Re
Runtime	At runtime (`LoadLibrary` + `GetProcAddress`)	No	**Not necessarily**	Hid beh → ma fav

Also: imports **by ordinal** (a number instead of a name) are a light obfuscation — map the number with a public ordinal table (common for ws2_32).

---

4. Packing — Structural Tells (read from the header, no execution)

Tell	Clean file	Packed file
Section names	`.text` / `.rdata` / `.data` / `.rsrc`	Odd names: `UPX0/1`, `LOL0/1`, random
Section raw vs virtual	Comparable	A section with **raw size 0** but large virtual size (unpack target)
Entropy	~4–6	A section near **7.9+** (compressed/encrypted body)
Section flags	code = exec, not writable	Section **executable *and* writable**
Import table	Full, descriptive	**Tiny stub**: `LoadLibrary`, `GetProcAddress`, `VirtualAlloc`, `VirtualProtect`, `ExitProcess`
Strings	Informative	**Almost none** / high-entropy garbage

Can you unpack it? Known packer (UPX → `UPX0/UPX1`) → reverse automatically (`upx -d`). **Disguised** known packer (UPX with sections renamed, e.g. `LOL0/LOL1`, and/or the `UPX!` marker stripped) → identify it by the **stub signature / entropy** in Detect It Easy, **restore the markers** (rename sections

back to `UPX0/UPX1`, put the `UPX!` magic back), then `upx -d` works. Genuinely **custom/unknown** packer → no auto-unpacker; unpack **manually** in a debugger: run to the Original Entry Point, dump memory, rebuild the IAT.

> **UPX disguise tell:** `upx -d` fails with *"file is modified/hacked/protected"* even though the layout (empty first section + high-entropy second section) and the stub epilogue (`… 58 61 8D4424 80 6A00 39C4 75FA 83EC80 E9 … jmp OEP`) are pure UPX → it is UPX wearing a mask, not a custom packer.

---

5. PE Header & Sections — Field Cheatsheet

Field	Normal	Flag when
Image base	`0x00400000` (exe)	—
Entry point	Inside `.text`	Points into an odd/high-entropy section → packing
Timestamp	Plausible date	Null/0 or absurd → stripped or fake (but any value is spoofable)
`.text`	exec, R	writable too → packing/self-modifying
`.data`	raw ≪ virtual (normal)	—
`.rdata`	raw ≈ virtual	—
`.rsrc`	small	**Very large** → embedded payload (dropper)
Exports in an .exe	none	Many exports → also meant to be loaded as a library

---

6. Strings — Categories to Hunt For

Category	Examples to expect	Meaning
Network endpoints	`1.2.3.4`, `:443`, `http://…`, domains	C2 address / URL (**check wide strings!**)
Shell / commands	`cmd.exe`, `COMSPEC`, `/c`, `del`, `> nul`	Command execution / self-delete

Category	Examples to expect	Meaning
Masquerade names	`wuauclt.exe`, `svchost.exe`, `*.dll`	Impersonating trusted processes
Persistence	`...\Services\...`, `Run` keys, `ServiceDll`, `ServiceMain`	Service / registry persistence
Beacons / protocol	`Hello.I am here!`, prompts, `%d` / `%s` format strings	C2 chatter / strings built at runtime
Encoding tables	64-char `A–Za–z0–9+/` (or `-/`) alphabet	Base64 / custom encoding of traffic
Dropped filenames	`Rasmon.dll`, `*.bat`, `VedioDriver.dll`	Files it writes to disk
Format strings	`http://%s`, `%s\%s`	URLs/paths assembled at runtime

> Imported function names also appear in strings — that is expected (the import table is bytes too), **not** a separate finding.

---

7. Host vs Network Signatures (how the exam questions split)

Host-based	Network-based
Dropped files (`Rasmon.dll`, `DFS.bat`)	Hard-coded IP / port
New service + registry keys	Domain / URL
Mutexes, processes spawned (`cmd.exe`)	Beacon strings, request structure
Self-deletion, timestomping, cleared logs	Proxy/`CONNECT`, user-agent

What usually "impedes analysis": packing (unpack it), runtime linking (resolve in a debugger), filenames/targets built at runtime (recover during dynamic analysis), encoding/encryption of traffic (decode in disassembly).
Procmon filter tip: filter `Process Name is <sample>` (and its children) to cut the noise; add `Operation is …` or a network filter to focus.

---

8. Consolidated IOC Tables — Lab 03 Samples

8.1 Identity & classification

Sample	MD5	Compiled	Packed?
Lab_03-1.exe	`4280aac55c1d3c327a6c00f0f0085677`	2009-05-14	No
Lab_03-2.exe	`02658bc9801f98dfdf167accf57f6a36`	2008-09-16	No
Lab_03-3.exe	`6c2a33512b8b0ec906982783e82b9678`	2011-11-08	**Yes — UPX disguised** (`LOL0/LOL1`, `UPX!` stripped)
Lab_03-4.exe	`9f880ac607cbd7cdfffa609c5883c708`	2009-07-20	**Yes — UPX**

SHA-256: 1 = `84b7967aad00e982842045e7b9744af0a457d46bba70456e5f99e7eb9cd783c7` · 2 = `8a35842d3f5963f715def0bbd0a53d7ffaae2d2ca79f56a5ac8bede64749d279` · 3 = `212b29a8e36ccc8f65205122e33d84940a156a9a91329f747a713f988a157948` · 4 = `a0dbee0e7238d028e4efd54fa24b94dc10c6a5f488b09ba49b37d90f22845b00`.

8.2 Network signatures

Sample	Indicator	Notes
Lab_03-1	`60.248.52.95:443`	C2 IP:port — **stored as a wide/UTF-16 string** (ASCII scan misses it); known IOC; 443 ⇒ likely TLS
Lab_03-1	`http://www.ueopen.com/test.html`	Embedded URL (leading `0x01` byte)
Lab_03-2	`69.25.50.10`	Hard-coded C2 IP

Sample	Indicator	Notes
Lab_03-2	`Hello.I am here!`	Beacon / check-in string
Lab_03-2	`CONNECT` , `HTTP/1.0`	Proxy tunnelling support
Lab_03-3	`http://%s` , `Mozilla` UA fragment	Hints leak while packed; full C2 artifacts recovered after `upx -d`
Lab_03-4	*(none at dropper stage)*	Network logic lives in dropped `Rasmon.dll`

8.3 Host signatures

Sample	Indicators
Lab_03-1	`cmd.exe` + `COMSPEC` + `/c del` (self-delete) · runtime-loaded `urlmon.dll` · base64-style alphabet (`…0123456789-/`) · prompt `*(SY)#`
Lab_03-2	Masquerades as `wuauclt.exe` · `\tasks\` staging dir · spawns `cmd.exe` · file read/write (`getf` / `putf` / `type`)
Lab_03-3	Few/no strings (**packing is the signature**) · renamed UPX sections `LOL0/LOL1` + UPX stub epilogue · stub imports `VirtualAlloc` / `VirtualProtect` / `LoadLibrary` / `GetProcAddress` · (after `upx -d` : full imports/strings of a network backdoor)
Lab_03-4	Drops `Rasmon.dll` (ServiceDll, exports `ServiceMain`) · service under `SYSTEM\CurrentControlSet\Services\…` · `svchost.exe -k SysIns` group · `DFS.bat` self-delete · timestomping · `ClearEventLog` · refs `VedioDriver.dll` , `acelpvc.dll` , `mdm.exe`

8.4 Key imports per sample

Sample	Signature imports
Lab_03-1	`socket` / `connect` / `send` / `recv` (by **ordinal**) · `ShellExecuteExA` `CreateProcessA` + `CreatePipe` + `PeekNamedPipe` · `GetComputerN`
Lab_03-2	Full WinINet chain (`InternetOpenA` → `HttpSendRequestA` → `Inter` `CreateProcessA` +pipes · C-runtime file I/O (`fopen` / `fread` / `fwri`
Lab_03-3	`LoadLibraryA` + `GetProcAddress` + `VirtualAlloc` + `VirtualProte` (stub) · leaked `InternetOpenA` , `WSAStartup`

Sample	Signature imports
Lab_03-4 (unpacked)	`FindResourceA` / `LoadResource` / `WriteFile` (drop) · `OpenSCManagerA` / `CreateServiceA` / `StartServiceA` · `RegCreate` `GetFileTime` / `SetFileTime` · `AdjustTokenPrivileges`

8.5 Persistence & purpose

Sample	Persistence	Purpose (one line)
Lab_03-1	None on disk (runs in memory, self-deletes)	Remote shell to attacker; encoded traffic
Lab_03-2	None explicit (tasking-based)	HTTP RAT: file transfer + command exec, posing as Windows Update
Lab_03-3	Recovered after UPX unpack	Network backdoor/downloader hidden under a disguised-UPX wrapper
Lab_03-4	**Windows service DLL under `svchost`** (auto-start at boot)	Dropper installing a persistent RAT with recon + anti-forensics